BESCHREIBUNG

Assemblerdokumentation

Cross-Assembler c82ass: Installation, Sprachdefinition, Anweisungen, Makros und Prüfbericht

Dieses PDF gibt die Quelle im Wortlaut wieder; Listen, Tabellen und Codeblöcke entsprechen der Quelle, weich umbrochene Zeilen sind zu Absätzen zusammengeführt.

Quelle	`C8205_Assembler_Dokumentation.md`	1.00-030
Stand	22.10.2026	
Umfang	31 Seiten, A4 hoch	
Erzeugt mit	`erzeuge_assemblerdokumentation_pdf.py`	

> Maßgeblich bleibt die Quelldatei. Dieses PDF wird nicht automatisch nachgeführt: Es ist aktuell, solange die Version oben mit der Versionszeile der Quelldatei übereinstimmt.

C 8205 – Assemblerdokumentation

Quelle: C8205_Assembler_Dokumentation.md · im Wortlaut der Quelle

Inhaltsverzeichnis

Cross-Assembler für C8205 / C8205Z – Version 1.00-030 4

1. Installation 4

1.1 Verzeichnisstruktur 4

2. Überblick 5

3. Der Assembler c82ass 6

3.1 Programmstart 7

3.1a Schalter --ABSPR (absolut adressierter KEP-Lochstreifen) 9

3.2 Sprachdefinition 10

3.3 Assembleranweisungen 11

3.3.1 Definiere Bereich 11

3.3.2 Definiere Bereich auf Null 11

3.3.3 Definiere Wert 11

3.3.4 Definiere Adresse 11

3.3.5 Definiere Text zur Ausgabe von Zeichen 11

3.3.6 Definiere Zeichen (spezielle Sonderzeichen) 12

3.3.7 Beginn des Quellcodes 12

3.3.8 Definition Relativadresse 13

3.3.9 Definition Absolutadresse 13

3.3.10 Definition Absolute Sektoradresse 13

3.3.11 Ende des verschieblichen Teils des Quelltextes 13

3.3.12 Pragma LEITADRESSE 13

Tabelle 2: Sonderzeichen für die dz-Anweisung 15

Tabelle 3: Ausgabe-Umschlüsselung außer Sonderzeichen dz 15

4. Der Makro-Präprozessor des Assemblers 16

4.1 Makros im Quelltext 18

5. Compilierungsbeispiel 19

5.1 Quelltext bsp_5_16.asm und Assemblerlauf 19

5.2 Durch c82ass erzeugter Objektcode (K1K2-Programmcode) für die Eingabe mit Kennzeichen (Format des Minimalprogramms) 20

6. Abgleich mit dem Assembler c82ass 20

6.1 Auffälligkeiten im Assembler 20

6.2 Angleichung an die erneuerte .odt (16.10.2026) 21

6.3 Änderungen vom 20.10.2026 (Symbol allein auf der Zeile, Vorspann, Usage, -o/-t) 21

6.4 Änderungen vom 21.10.2026 (Version 1.00-010: Warnungen, Schalter -m, --xref, Build) 21

6.5 Änderungen vom 21.10.2026 (Pufferüberläufe, Fehlerorte für Pass 0 bis 2) 22

6.6 Änderungen vom 21.10.2026 (Version 1.00-011: keine -Wshadow-Hinweise mehr) 23

6.7 Änderungen vom 21.10.2026 (Version 1.00-012: Fehlerorte, Zeilengrenze, BOM) 23

6.8 Änderungen vom 21.10.2026 (Version 1.00-013: Überlauf bei leerem dt, zwei Warnungen) 23

6.9 Änderungen vom 21.10.2026 (Version 1.00-014: pass3 zerlegt) 24

6.10 Änderungen vom 21.10.2026 (Version 1.00-015: pass2 und loeseMacroAuf zerlegt, Speicherfehler behoben) 25

6.11 Änderungen vom 21.10.2026 (Version 1.00-016: da mit nicht definiertem Symbol) 25

6.12 Änderungen vom 21.10.2026 (Version 1.00-017: Fehlerort bei Fehlern in der Makrodatei, Prüfskript, CI) 26

6.13 Änderungen vom 21.10.2026 (Version 1.00-018: Makro-Modul herausgelöst) 26

6.14 Änderungen vom 21.10.2026 (Version 1.00-019: da verlangt eine relative Adresse) 27

6.15 Änderungen vom 21.10.2026 (Version 1.00-020: Quelltext in Module zerlegt, pass0, pass1 und main zerlegt) 27

6.16 Änderungen vom 21.10.2026 (Version 1.00-021: zentrale Fehlerausgabe mit Obergrenze) 28

6.17 Änderungen vom 21.10.2026 (Version 1.00-022: projekdetails abgesichert, Referenz bereinigt, Makefile) 28

6.18 Änderungen vom 21.10.2026 (Version 1.00-023: Rekursionsfehler ohne Abbruch) 29

6.19 Änderungen vom 21.10.2026 (Version 1.00-024: Warnungen für Sprungziele und nicht erreichbaren Code) 29

6.20 Änderungen vom 21.10.2026 (Version 1.00-025: Berichte im .log, Rekursionsfehler mit Ort, make fuzz) 30

6.21 Änderungen vom 21.10.2026 (Version 1.00-026: Opcode-Tabelle entfernt, --xref mit Makroexpansionen) 30

6.22 Änderungen vom 21.10.2026 (Version 1.00-027: Schalter --ABSPR) 30

6.23 Änderungen vom 22.10.2026 (Version 1.00-028: Warnung bei --ABSPR mit LEITADRESSE 00 und Startadresse) 31

6.24 Änderungen vom 22.10.2026 (Version 1.00-029: Warnung zur KEP-Bahn eingeschränkt) 31

6.25 Änderungen vom 22.10.2026 (Version 1.00-030: Regressionstest für --ABSPR) 31

C 8205 – Assemblerdokumentation

Quelle: C8205_Assembler_Dokumentation.md · im Wortlaut der Quelle

Version: 1.00-030

Diese Datei ist eine Textableitung von `C8205_Assembler_Dokumentation.odt` in der Version 1.00-030 (per `extract-text`), gedacht zum Durchsuchen und Zitieren. **Maßgeblich bleibt die .odt** (siehe `C8205-Projektstruktur.md`, Abschnitte 1 und 3); bei Widersprüchen gilt die .odt. Diese Ableitung wird nicht automatisch aktualisiert, wenn die .odt sich ändert.

Stand 22.10.2026 (.odt nachgeführt): Die .odt trägt jetzt Version 1.00-030 und enthält Usage mit `-WARN=SYM,SPR,TOT,ALLE` und `--ABSPR`, die Dateiendung `.kep`, Abschnitt 3.1a, die neue Regel für `da`, den Wegfall der Mnemonics, die Bezeichnung „Eingabe mit Kennzeichen (Format des Minimalprogramms)“ (Pass 3, `equ`, 5.2), die Quelltextliste (1.1), die Pragma-Verweise auf 3.3.12 und die Einträge 1.00-018 bis 1.00-030 im Änderungsverzeichnis (Abschnitt 6). Die Abschnitte 6.1 bis 6.22 dieser Ableitung sind eine eigene Übersicht und stehen in der .odt in Kurzform.

Stand 22.10.2026 (Ableitung auf Version 1.00-030 gesetzt, die .odt hat Vorrang): Versionsangaben in Titel, Kopf, Usage (2) und `--version` (3.1) von 1.00-027, 1.00-026 und 1.00-010 auf 1.00-030 gesetzt; `--ABSPR` in Usage-Zeile und Optionsliste (2) und der Zusatz „sofern nicht mit `-m` ein Pfad angegeben ist“ (3.1) aus der .odt übernommen; Abschnitte 6.23 bis 6.25 für 1.00-028 bis 1.00-030 ergänzt. Das Beispiel in 5.1 zeigt wie in der .odt die Versionszeile „Version 1.00“ (Mitschnitt eines älteren Laufs); der Assembler gibt heute „1.00-030“ aus. Das PDF `C8205-Assemblerdokumentation_A4.pdf` ist aus dieser Fassung neu erzeugt.

Stand 22.10.2026 (Ableitung, Entscheidung des Projektinhabers): Die Bezeichnung „Kleines Eingabeprogramm" für den Objektcode (`.obj`) in Pass 3 (Abschnitt 1), bei `equ` (3.3.5) und in der Überschrift von 5.2 ist ersetzt durch „Eingabe mit Kennzeichen (Format des Minimalprogramms)": Das KEP liest nur den binären Lochstreifen `.kep` (3.1a, `--ABSPR`). Die .odt enthält dieselbe Bezeichnung inzwischen (Stand 1.00-030); der Hinweis „In der .odt nachzuführen“ ist damit erledigt.

Stand 20.10.2026 (Ableitung nachgeführt nach der .odt vom 20.10.2026): Die .odt beschreibt jetzt die Schalter `-o` und `-t` sowie `-h`, `--help` und `--version` mit der vollständigen Usage-Ausgabe (Abschnitt 2), den Vorspann der `.test`-Datei aus dem Quelltext (3.1), das Symbol allein auf einer Zeile (3.2) und das Label allein vor `start` (3.3.7). `-WARN=SYS` ist in `-WARN=SYM` berichtigt, denn der Assembler kennt nur `SYM`. Abschnitt 6.3 hält die zugehörigen Änderungen an `ca82ass.c` fest. Das PDF `C8205-Assemblerdokumentation_A4.pdf` ist nicht nachgeführt.

Stand 19.10.2026 (Ableitung neu abgeglichen mit der .odt vom 16.10.2026, Entscheidung des Projektinhabers: OP-23 „aus der .odt ableiten“): Der Satz zu `node.js` ist entfallen (steht in der .odt nicht mehr); `-WARN=SYS` (1.1) und die Anweisung `spa` (neuer Abschnitt 3.3.10, damit „Ende des verschieblichen Teils“ = 3.3.11 und „Pragma LEITADRESSE“ = 3.3.12) sind aus der .odt übernommen. **Die .odt verweist an mehreren Stellen auf „3.3.11“ für das Pragma, obwohl dort 3.3.12 steht; diese Ableitung führt die richtigen Verweise (3.3.12).** Das PDF `C8205-Assemblerdokumentation_A4.pdf` ist nicht geändert und nennt `node.js` noch.

Stand 21.10.2026: Die .odt ist auf Version 1.00-010 nachgeführt (Usage mit `-m`, `--xref`, `--color`/`--no-color`; unbekannte Optionen als Fehler; Fehlerort `<datei>:<zeile>:`; keine `.obj`/`.mqo`/`.test` bei Fehlern; neuer Abschnitt 6 „Änderungsverzeichnis“ in der .odt, in dieser Ableitung als 6.4). Das PDF ist nicht nachgeführt. In 1.00-011 ist `-Wshadow` bereinigt (6.6). In 1.00-012 sind Fehlerorte und Grenzen nachgeführt (6.7). In 1.00-013 sind ein Überlauf bei leerem `dt`-Operanden behoben und zwei Warnungen ergänzt (6.8). In 1.00-014 ist `pass3` zerlegt (6.9). In 1.00-015 sind `pass2` und `loeseMacroAuf` zerlegt und mehrere Fehler in der Makro-Auswertung behoben (6.10). In 1.00-016 meldet `da` ein nirgends definiertes Symbol als Fehler (6.11). Die .odt ist auf 1.00-016 gesetzt (Versionsangaben und Änderungsverzeichnis); die Fehlertabelle in 3.1 und die Zeilengrenze in 3.2 sind dort noch nicht eingearbeitet (Text in 6.8).

Stand 16.10.2026: Die Abschnitte 1 bis 5 stimmen inhaltlich mit der erneuerten .odt (Version 1.00-005, zuletzt gespeichert am 16.10.2026) überein. Neu gegenüber der vorigen Ableitung (Stand 15.10.2026): Der `<adressausdruck>` (3.2) nennt keine Obergrenze für die Zahl der Symbole mehr (die Zeile „nur maximal ein Symbol erlaubt“ ist entfallen). Beim `<werteausdruck>` (3.2) und bei `dw` (3.3.3) sind paarweise Differenzen symbolischer Adressen zulässig, auch mehrfach. Die Verzeichnisstruktur (1.1) ist gekürzt (Beispielprojekte nur noch `Projekt_1`). In 3.3.8 entfällt der Satz zum Pragma `LEITADRESSE`; die Ladeadresse existiert dort nur beim Laden des Objektcodes.

Rechtschreibung, Anführungszeichen und offensichtliche Tippfehler sind bereinigt; Überschriften, Codeblöcke und Tabellen haben die Form der Markdown-Ableitung; einzelne Zusätze der Ableitung (Tabellenköpfe, Beispiele und Erläuterungen in Klammern) ändern den Inhalt der .odt nicht. Die Anweisung `dw <literal>` beschreibt die .odt nicht; sie ist deshalb auch in Abschnitt 3.3.3 dieser Ableitung nicht beschrieben (der Assembler weist sie ab, s. Abschnitt 6). Abschnitt 6 stammt nicht aus der .odt.

Cross-Assembler für C8205 / C8205Z – Version 1.00-030

Die gesamte Software läuft sowohl unter Linux als auch Windows 10. Sie ist in C (gcc) programmiert.

1. Installation

Zum Zweck der Installation ist die Datei `C8205.zip` in einem geeigneten Verzeichnis zu entpacken. Beim Entpacken wird ein Unterverzeichnis angelegt, in dem sich sowohl Windows- als auch Linux-Binaries befinden.

1.1 Verzeichnisstruktur

Der Assembler liegt im Unterverzeichnis `Assembler` des Gesamtprojekts C8205. Es enthält folgende Struktur (Beispielprojekte gekürzt):

```
make_c82ass.sh                # Skript zum Erzeugen des Assemblers unter Linux
c82ass.sh                     # Ausführen des Assemblers
asm_alle.sh                   # assembliert alle .asm-Dateien unterhalb eines Verzeichnisses (meldet nur Fehler)
asm_aufraeumen.sh             # entfernt .log, .mqo, .obj und .sym (Trockenlauf; --loeschen löscht)
make_c82ass.bat               # Batch-Datei zum Erzeugen des Assemblers unter Windows (Pendant zu make_c82ass.sh)
Dokumentation                 # diese Dokumentation
_Linux                        # Linux-Binary des Assemblers
projekdetails                 # zuletzt benutzte Aufrufparameter
projekdetails.<Projekt>       # gesicherte Aufrufparameter je Projekt
Projekt_1                     # Projektverzeichnis (Beispiel, mit Unterverzeichnissen)
Sourcecode                    # C-Quellcode des Assemblers selbst
_Windows                      # Windows-Binary des Assemblers
```

```
./Assembler/Dokumentation:
C8205_Assembler_Dokumentation.md   C8205_Assembler_Dokumentation.odt   C8205_Assembler_Dokumentation.pdf

./Assembler/_Linux:
c82ass

./Assembler/Projekt_1:
bsp_5.15
bsp_5.16
bsp_seite_39
bsp_seite_40
bsp_seite_42
… --weitere ---

./Assembler/Sourcecode:
c82_austausch.c        c82_austausch.h  ca82_intern.h
ca82_main.c        ca82_quelle.c      ca82_listing.c     ca82_zustand.c
ca82_pass0.c       ca82_pass1.c       ca82_pass2.c       ca82_pass3.c
ca82_ausdruck.c    ca82_codierung.c   ca82_testdatei.c   ca82_hilfen.c
ca82_fehler.c      ca82_fluss.c       ca82_fluss.h
ca82_protokoll.c  ca82_protokoll.h  ca82_standards.c  ca82_standards.h
ca82_berichte.c    ca82_berichte.h    ca82_symbole.c     ca82_symbole.h
ca82_makros.c      ca82_makros.h      ca82_typen.h       ca82_version.h
c82ass.makros

./Assembler/_Windows:
```

Projektverzeichnisse können beliebig benannt sein, auch Unterverzeichnisse enthalten. In ihnen steht der Quelltext, die erzeugte Objektdatei (die K1K2-ladefähige Textdatei) und weitere Hilfsdateien. Die Programme `c82ass`, `c82load`, `c82dump`, `c82simi` erwarten ihre Dateien immer in einem Projektunterverzeichnis, das im gleichen Pfad wie sie selbst liegen muss.

In der vorliegenden Projektstruktur ist nur `c82ass` enthalten. Die Programme `c82load`, `c82dump`, `c82simi` und der `C8205Simulator` sind nicht Teil dieses Projekts und liegen nicht bei; sie werden in dieser Dokumentation nur erwähnt, weil `c82ass` Dateien für sie erzeugt (`.mqo` und `.obj` für `c82load`, `.mqo` für `c82simi`, `.sym` für `c82dump`). Für die Prüfung von Assemblerergebnissen im Interpreter dient stattdessen die Datei `.test` (s. `C8205-Interpreter-Durchführung_eines_Tests.md`, Abschnitt 2a).

`make_c82ass.sh` ist ein Shell-Script zum Erzeugen des Assemblers. Die Datei `make_c82ass.bat` leistet das Gleiche für den MinGW-w64-Crosscompiler unter Windows. Der Pfad `_Linux` enthält alle Linux-Binaries und Libraries, der Pfad `_Windows` die Pendants für Windows 10. `Sourcecode` enthält ausschließlich den C-Quellcode.

`asm_alle.sh [Basisverzeichnis]` assembliert alle `.asm`-Dateien unterhalb eines Verzeichnisses (Vorgabe: `./`) mit `c82ass.sh`. Bei fehlerfreiem Lauf erscheint keine Meldung; bei einem Fehler wird die Datei mit den Fehlerzeilen des Assemblers gemeldet. Rückgabewert: 0 = alles fehlerfrei, 1 = Fehler. `asm_aufraeumen.sh` entfernt die erzeugten

Dateien `.log`, `.mqo`, `.obj` und `.sym` unterhalb von `Assembler` (nicht in `Dokumentation`, `Sourcecode`, `_Linux`, `_Windows`); ohne Parameter wird nur angezeigt, was gelöscht würde (Trockenlauf), mit `--loeschen` wird tatsächlich gelöscht. Beide Skripte gehören in das Verzeichnis `Assembler`.

2. Überblick

Sowohl das Programm `c82ass.sh` (Linux) als auch das Windows-Binary `_Windows\c82ass.exe` (die Datei `make_c82ass.bat` erzeugt es nur) werden über Aufrufparameter versorgt. Bei jedem Start eines der Programme werden die jeweils letztbenutzten Parameter automatisch in der Datei `projekdetails` abgelegt, sodass ein Folgestart ohne Angabe von Parametern gelingt. Zur Änderung müssen die Parameter einfach mit veränderten Werten wieder angegeben werden.

```
$ c82ass.sh -h
C8205 Assembler C82aas Version 1.00-030
Usage: ./_Linux/c82ass -p Projektverzeichnis -s Quellcode-Datei [-o Objektdatei] [-t Testdatei] [-m Makrodatei]
    [-WARN=SYM,SPR,TOT,ALLE] [--xref] [--ABSPR] [--color | --no-color]
       ./_Linux/c82ass -h | --help | --version

  -p Projektverzeichnis  Verzeichnis mit dem Quelltext; dorthin werden auch die Ausgabedateien
                         (.log .obj .mqo .sym .dmp .test) geschrieben. Ohne -p gilt der zuletzt
                         benutzte Wert aus der Datei 'projekdetails' im aktuellen Verzeichnis.
  -s Quellcode-Datei     Assembler-Quelltext im Projektverzeichnis; die Endung .asm darf entfallen.
                         Ohne -s gilt ebenfalls der zuletzt benutzte Wert aus 'projekdetails'.
  -o Pfad | Datei        Ablageort der .obj-Datei statt des Projektverzeichnisses. Verzeichnis
                         (existiert, oder endet auf / bzw. \): Dateiname aus -s mit Endung .obj.
                         Sonst Dateiname mit oder ohne Pfad (relativ zum aktuellen Verzeichnis),
                         unveraendert uebernommen. Das Verzeichnis muss existieren.
  -t Pfad | Datei        wie -o, aber fuer die .test-Datei (Endung .test).
  -m Makrodatei          Pfad der Makrodatei statt 'c82ass.makros' (sonst: aktuelles Verzeichnis,
                         ersatzweise 'Sourcecode/'). Mit -m wird nur dieser Pfad benutzt.
  -WARN=liste            Zusaetzliche Warnungen; Werte durch Komma getrennt (Gross-/Kleinschreibung egal):
                           SYM  Symbole, die definiert, aber nirgends verwendet werden.
                           SPR  Sprungbefehle, deren Ziel ein Datenwort (dw, da) oder ein unbelegtes Wort ist.
                           TOT  Befehle hinter einem unbedingten Sprung, auf die nichts verweist.
                           ALLE alle drei.
                         Die Meldungen stehen auch im .log (hinter der Symboltabelle).
  --xref                 Gibt nach fehlerfreiem Lauf eine Querverweisliste aus: je Symbol Wert,
                         Definitionszeile und alle Zeilen des Quelltexts, die es verwenden. Sie steht auch im .log.
  --ABSPR                Statt der .obj-Datei einen absolut adressierten, binaer im R300-Code codierten
                         Lochstreifen <name>.kep fuer das Kleine Eingabeprogramm (KEP) erzeugen:
                         Leitbefehl <start> 240: und Worte als Dualziffern (auch --ABSOLUTPROGRAMM, --ASBPR).
  --color, --no-color    Farbige Ausgabe erzwingen bzw. abschalten. Ohne Angabe nur, wenn STDOUT und
                         STDERR ein Terminal sind und die Umgebungsvariable NO_COLOR fehlt.
  -h, --help             Diese Hilfe ausgeben.
  --version              Die Versionsnummer ausgeben (steht auch in der ersten Ausgabezeile).
```

Im Usage-Text steht der Aufrufname (`argv[0]`); über `c82ass.sh` erscheint daher `./_Linux/c82ass`. `c82ass.sh` ruft `./_Linux/c82ass` relativ auf und muss deshalb aus dem Verzeichnis `Assembler` gestartet werden; dort werden auch `projekdetails` und (zuerst) `c82ass.makros` gesucht (sofern nicht mit `-m` ein Pfad angegeben ist).

- `-p` nennt das Verzeichnis, in dem die `.asm`-Datei **direkt** liegt (z. B. `Projekt_1/bsp_5.16`). Unterverzeichnisse werden nicht durchsucht; ein Pfadanteil bei `-s` wird abgeschnitten.
- Hat der Name bei `-s` keine Endung aus genau drei Zeichen, wird `.asm` angehängt (`bsp_5_16` wird zu `bsp_5_16.asm`).
- `-o` und `-t` legen fest, wo die Objektdatei (`.obj`) bzw. die Testdatei (`.test`) abgelegt wird. Ohne diese Schalter liegen beide im Projektverzeichnis; der Dateiname wird aus `-s` gebildet. Ist ein Schalter angegeben, gilt statt des Standardorts der angegebene Pfad. Der Wert ist entweder ein Verzeichnis (es existiert bereits, oder der Wert endet auf `/` bzw. `\`) – dann wird der Dateiname aus `-s` gebildet (Name der Quellcode-Datei ohne `.asm`, dazu `.obj` bzw. `.test`) – oder ein Dateiname, mit oder ohne Pfad. Ein Dateiname wird unverändert übernommen (eine fehlende Endung wird nicht ergänzt); ohne Pfadanteil gilt er relativ zum aktuellen Verzeichnis, nicht zum Projektverzeichnis.
- Das Verzeichnis muss existieren; der Assembler legt es nicht an. Ein nicht vorhandenes Verzeichnis, ein Schalter ohne Wert (auch wenn der nächste Parameter mit `-` beginnt) und ein zu langer Pfad sind Fehler (Rückgabecode 16). `-o` und `-t` gelten nur für den einen Aufruf und werden nicht in `projekdetails` gespeichert; die übrigen Ausgabedateien (`.log`, `.mqo`, `.sym`, `.dmp`) bleiben im Projektverzeichnis. Beispiel: `$ c82ass.sh -p Projekt_1/bsp_5.16 -s bsp_5_16 -o ausgabe/ -t tests/bsp_5_16_neu.test`
- `-h` und `--help` geben den Usage-Text aus, `--version` nur die Versionszeile „C8205 Assembler C82aas Version 1.00-030“. Beide enden ohne Assemblerlauf mit Rückgabecode 0.

- `projekdetails` enthält die Zeilen `-p <Verzeichnis>` und `-s <Datei>`; fehlt einer der Parameter im Aufruf, wird der gespeicherte Wert benutzt. Beim Wechsel des Projektverzeichnisses wird die bisherige Datei als `projekdetails.<altes Projekt>` gesichert (Schrägstriche im Namen werden zu _, z. B. `projekdetails.Projekt_1_bsp_5.16`) und die gesicherte Datei des neuen Projekts, falls vorhanden, zurückgeholt.
- Rückgabecode: 0 = fehlerfrei, 16 = Fehler (jede Art von Abbruch). Parameter ab einem Argument, das mit > beginnt, werden ignoriert.
- Unbekannte Optionen (z. B. `-x`) werden nicht mehr stillschweigend übergangen: Der Assembler meldet „Fehler: Unbekannte Option …“, gibt den Usage-Text aus und endet mit Rückgabecode 16. Ein Argument ohne führenden Bindestrich, das keinen Wert zu einem Schalter bildet, ergibt nur eine Warnung und wird ignoriert.
- `-WARN=<liste>` schaltet zusätzliche Warnungen ein. Die Werte stehen durch Komma getrennt (z. B. `-WARN=SYM,SPR`); Groß-/Kleinschreibung ist egal, ein unbekannter Wert ergibt eine Warnung. Über `c82ass.sh` darf auch `--WARN=SYM` geschrieben werden. Alle Warnungen erscheinen erst nach fehlerfreiem Pass 3 und verändern weder `.obj` noch `.dmp`; sie lassen den Rückgabecode bei 0. In die `.log`-Datei werden sie (ab 1.00-025) hinter der Symboltabelle angehängt, ohne Farbcodes und mit denselben Überschriften wie auf STDOUT; ohne `-WARN=` und `--xref` bleibt die `.log` unverändert.
  - `SYM` meldet Symbole, die definiert, aber nirgends verwendet werden (vom Assembler erzeugte Symbole wie Literalsymbole und Makro-Labels werden nicht gemeldet).
  - `SPR` meldet Sprungbefehle (unbedingt, bedingt oder mit Rückkehrabsicht; Operationscodes 000–007, 024–027, 040–057, 064–077, 100–107, 124–127, 140–157, 170–177), deren Ziel kein Befehl ist, sondern ein Datenwort (`dw` mit Wert ungleich 0, `da`) oder ein Wort, das der Assembler gar nicht belegt. Nicht gemeldet werden Ziele, die als Befehl dienen können: `dz`, `dt`, `dto` und `dtm` erzeugen Ausgabebefehle, `db`, `dbn` und `dw 0` reservieren Zellen, in die das Programm zur Laufzeit etwa den Rücksprungbefehl schreibt. Ebenfalls nicht gemeldet wird ein Sprungbefehl, den ein Lade- oder Speicherbefehl anspricht (Adresskonstante in Befehlsform wie `000 feld`): Er gilt als Datenzelle. Sprünge mit absoluter Adresse (`abs`-Symbol) werden nicht geprüft.
  - `TOT` meldet Befehle, die unmittelbar hinter einem unbedingten Sprung (000–007, 100–107) stehen und auf die nichts verweist: keine Marke, die irgendwo verwendet wird, kein Operand mit relativer Adresse, kein `da`-Wort, nicht die Startadresse und nicht Adresse 0000. Folgen mehrere solche Befehle aufeinander, wird nur der erste gemeldet, mit der Zahl der folgenden. Nicht gemeldet wird ein unbedingter Sprung hinter einem unbedingten Sprung (Sprungtabelle, die das Programm durch berechnete Adressen anspringt; die C8205 kennt keinen indirekten Sprung) und ein Befehl hinter einem Sprung, den ein Lade- oder Speicherbefehl anspricht (das Programm überschreibt ihn zur Laufzeit).
  - `ALLE` schaltet `SYM`, `SPR` und `TOT` ein.
  - `SPR` und `TOT` sind Hinweise und keine Fehler: Programme, die Befehle zur Laufzeit in Datenworte schreiben oder Sprungadressen berechnen, können sie zu Recht auslösen. Die Meldungen haben die Form `<datei>:<zeile>: Warnung: …` auf STDERR; auf STDOUT stehen eine Überschrift und die Anzahl.
- `-m <Makrodatei>` gibt den Pfad der Makrodatei an, statt des festen Namens `c82ass.makros`. Mit `-m` wird nur dieser Pfad benutzt (keine Ersatzsuche in `Sourcecode/`). Lässt sich die Datei nicht öffnen, endet der Lauf mit Rückgabecode 16 und einer Meldung mit Pfad und Ursache; `-m` ohne Wert ist ebenfalls ein Fehler (16).
- `--xref` gibt nach einem fehlerfreien Lauf eine Querverweisliste aus: je Symbol Wert, Definitionszeile (Name in Spalte 1) und alle Zeilen des Quelltextes, in denen der Name als ganzes Wort vorkommt (Kommentare ab ; zählen nicht). Vom Assembler erzeugte Symbole fehlen. Die Operanden aus Makroexpansionen stehen im Listing und zählen mit: ein Symbol, das ein Makro verwendet, gilt als verwendet in der Zeile des Makroaufrufs (Definitionen im Quelltext und in der Makrodatei zählen nicht gesondert). Die Liste steht zusätzlich am Ende der `.log`-Datei. Ohne `--xref` entsteht keine Liste.
- `--color` und `--no-color` erzwingen bzw. unterdrücken die farbige Ausgabe. Ohne Angabe wird nur farbig ausgegeben, wenn STDOUT und STDERR ein Terminal sind und `NO_COLOR` nicht gesetzt ist; bei Umleitung in Datei oder Pipe stehen keine Farbcodes in der Ausgabe.

3. Der Assembler c82ass

Version: 1.00

3.1 Programmstart

Der Assembler `c82ass` erwartet eine Eingabedatei mit `<quelltext>` unter einem Dateinamen `.asm`.

- **Pass 0:** liest die Makrodatei `c82ass.makros` (Vorgabe `c82ass.makros`; gesucht zuerst im Arbeitsverzeichnis des Aufrufs, sonst in `Sourcecode/`; mit `-m` ist ein anderer Pfad wählbar), sammelt die im Quelltext definierten Makros (Abschnitt 4.1), prüft die Makrodefinitionen und löst die Makroaufrufe im Quelltext auf (Abschnitt 4).
- **Pass 1:** ermittelt die Literaltabelle, ausgegeben im Logfile `.log`.
- **Pass 2:** ermittelt die Symboltabelle, ausgegeben im Logfile `.log`, und schreibt die Symboldatei `.sym`.
- **Pass 3:** erzeugt aus dem aufgelösten Quelltext (Literale und Symbole eingesetzt) die Kennzeichenbefehle für die Eingabe mit Kennzeichen (Format des Minimalprogramms, nicht des KEP), ausgegeben im Objektfile `.obj`. Im selben Pass entstehen die Hilfsdatei `.mqo`, die formatierte Speicherbelegung `.dmp` und die Trommeldatei `.test` für den Interpreter.

Auf STDOUT gibt der Assembler den formatierten Quelltext (einschließlich der aufgelösten Literale) aus. Der Aufruf erfolgt zum Beispiel in der Form:

```
$ c82ass.sh -p PzBeispiele_4 -s bsp_5.17_Oktaldruck.asm
```

Meldungen mit `Fehler:`, `Warnung:` und `Abbruch` sowie die Statuszeilen `Projektverzeichnis:`, `Benutze für -s die Datei:`, `Leitadresse:` (nur wenn der Quelltext das Pragma `LEITADRESSE` enthält, siehe 3.3.12) und `Startadresse:` gehen auf STDERR (farbig hervorgehoben). Fehler werden je nach Pass in unterschiedlicher Form gemeldet (siehe die folgende Tabelle); jeder Fehler beendet den Lauf mit Rückgabecode 16. Fehlermeldungen mit Ort erscheinen in der Form `<datei>:<zeile>: Fehler: <text>`. Bei einem Fehler werden keine `.obj`-, `.mqo`- und `.test`-Datei hinterlassen (eine unvollständige Objektdatei könnte sonst versehentlich geladen werden); `.log`, `.sym` und `.dmp` bleiben zur Fehlersuche erhalten. Meldungen und Statuszeilen sind nur auf einem Terminal farbig (siehe `--color`).

Fehlerausgaben

Fehlerart	Ausgabe auf STDOUT (Listing)	Ausgabe auf STDERR	Ablauf danach
Pragma `LEITADRESSE` fehlerhaft (beim Einlesen des Quelltextes, vor Pass 0; siehe 3.3.12)	–	`Fehler: Zeile <n> der Quelldatei: Das Pragma LEITADRESSE …`, `Fehler: Abbruch.`	Abbruch, keine Ausgabedateien
Makrodefinition fehlerhaft (Pass 0)	–	`Fehler: Makrodefinition …`, `Fehler: N fehlerhafte Makrodefinition(en) …`	Abbruch nach Pass 0, keine Ausgabedateien
Makroaufruf fehlerhaft (Pass 0), z. B. Parameteranzahl oder -typ	`;*** Fehler: erwartet 2 Parameter, aber 1 wurden übergeben!` bzw. `;*** Fehler $1 Parameter $1: Wert "123" …`	`Warnung: Compilierung endete mit FEHLERn.`	Abbruch nach Pass 0, keine Ausgabedateien
Syntaxfehler in Pass 1 (z. B. `#` am Anfang eines `dt`-Textes)	`;*** 1 Fehler: …`	`Fehler: bei Operand: … in Zeile: …`, `Fehler: Abbruch wegen Syntaxfehler.`	Pass 2 läuft noch (`.log`, `.sym`), Pass 3 entfällt
Fehler in Pass 2 (z. B. Symbol doppelt, Programmzählerüberlauf über 7777)	`;*** (2) Fehler: …`	`Fehler: Abbruch wegen Syntaxfehler.`	Pass 3 entfällt
Fehler in Pass 3 (Operand oder Operationscode ungültig)	Zeile `<adresse>: EE …` mit darunter stehender Meldungszeile `bei Operand: … ungültig`	`<datei>:<zeile>: Fehler: Operationscode … ist ungültig` bzw. `Symbol … ist nicht definiert`; `Fehler: Abbruch. Keine Objekt-, mqo- und Testdatei erzeugt …`	alle Pässe gelaufen; keine `.obj`, `.mqo`, `.test`; `.log`, `.sym`, `.dmp` bleiben
Belegungsfehler in Pass 3	`;*** 3 Fehler: Adresse … doppelt belegt: …`	`Fehler: Adresse … doppelt belegt: …`, `Fehler: Abbruch.`	wie Fehler in Pass 3

Belegungsprüfung (Pass 3): Jedes Wort der Trommel darf nur von einer Anweisung beschrieben werden. Geprüft werden Maschinenbefehle, `dw`, `da`, `dbn`, `dt`, `dto`, `dtm`, `dz` und die automatisch erzeugten Literale. Überschneidet sich die

fortlaufende Speicherbelegung mit einem früher belegten Bereich, zum Beispiel weil ein `equ` zurückspringt oder das Programm in einen mit `equ` weiter hinten angelegten Datenbereich hineinläuft, meldet der Assembler `Adresse <oktal> doppelt belegt: "<neue Zeile>" ueberschneidet sich mit "<frühere Zeile>".` und der Lauf endet mit Rückgabecode 16. Pro Zeile wird höchstens ein Konflikt gemeldet. Nicht geprüft werden `db` (reserviert nur Speicherplätze), `equ`, `start`, `abs` und `end`.

Die `<adresse>` der Befehle im Objektfile beginnt immer bei 0000. Ein Operand von `start` ändert die Adressen nicht; der Wert steht nur in der Kennzeichenzeile `21 0000 <zahl>` (siehe 3.3.7).

Der Assembler erzeugt bzw. bearbeitet folgende Dateien im Verzeichnis der `.asm`-Datei (Beispiel `Projekt_1/bsp_5.16/`):

Endung	Inhalt
`.asm`	Quelltext (Eingabe)
`.dmp`	formatierte Speicherbelegung (Listing; das Kennzeichen 01 kann durch das Pragma `LEITADRESSE` ersetzt sein, siehe 3.3.12): je Zeile Adresse mit Maschinenwort (`<adresse>: <K1> [<K2>] [<Wert>]`), Label, Mnemonic, Operand und Kommentar in Spalten. Es ist dasselbe Listing, das der Assembler auf STDOUT ausgibt (Meldung „Ausgabe der formatierten Speicherbelegung nach: …“), jedoch ohne die Farbcodes der Fehlermeldungen
`.log`	Abschnitte, soweit vorhanden: Literaltabelle, EQU-Definitionen, ABS-Definitionen, Symboltabelle (Symbol, RelAddr, Adresse, original Operation) und Adresstabelle (nach Adresse sortiert)
`.mqo`	aufbereiteter Quelltext für `c82load` (kombiniert ihn mit dem Maschinencode); auch für `c82simi`, das ggf. bei der Einzelschrittverarbeitung die Kommentare einblendet. Gegenüber dem Quelltext fehlen `start`, `end`, Makrodefinitionen, reine Kommentarzeilen und `abs`-Anweisungen; Makroaufrufe sind durch ihre Ersetzung ersetzt, `equ`-Operanden aufgelöst, und die Literale stehen am Ende als `dw`-Zeilen (`l0001 dw 5`)
`.obj`	mit `c82load` ladefähiger Objektcode K1K2; das Kennzeichen 01 kann mit dem Pragma `LEITADRESSE` durch 00 oder 02 … 09 ersetzt werden (siehe 3.3.12); bei 00 entfällt außerdem die Kennzeichenzeile `21 0000 <zahl>`
`.kep`	nur mit dem Schalter `--ABSPR`, **statt** `.obj`: absolut adressierter, binär im R300-Code codierter Lochstreifen für das Kleine Eingabeprogramm (KEP); siehe 3.1a
`.sym`	Symboldatei; wird von `c82dump` benutzt, um zum Trommeldump Symbole zuzuordnen. Je Zeile `<symbol>,<adresse>` (vierstellig oktal, in Quelltextreihenfolge, einschließlich Literalsymbolen wie `l0001`)
`.test`	Trommeldatei im Format des Interpreters (`Adresse: Wert # Quelltextzeile`) mit Kopf (Abschnitt `Trommeldatei:`) und Fuß (Abschnitt `Tastenfolge:` mit `H G R`). Der Kopf besteht aus den Vorspannzeilen des Quelltextes (siehe unten); enthält der Quelltext keine, erzeugt der Assembler als Platzhalter `#DOK-0001`, `#SPEI-0001 0000-0000` und `#PROT STUFE=2`. Kopf und Fuß müssen für den jeweiligen Fall ergänzt bzw. erweitert werden. Zeilen gibt es nur für Wörter mit K1=00 oder 01 (Befehle, `dw`, `da`, `dz`, `dt`, `dto`, `dtm`, Nullwort); `db`, `equ`, `start` und `abs` erzeugen keine Zeile; Grundlage für Testaufträge, s. `C8205-Interpreter-Durchführung_eines_Tests.md`, Abschnitt 2a

Vorspann der `.test`-Datei: Der Abschnitt `Trommeldatei:` kann im Quelltext vorgegeben werden. Dazu stehen die gewünschten Zeilen als Kommentarzeilen im Quelltext: Das erste Zeichen außer Leerraum ist ein Semikolon, danach folgt (auch nach Leerraum) eine Testmarke. Der Assembler übernimmt jede solche Zeile ohne das führende Semikolon und den Leerraum dahinter in der Reihenfolge des Quelltextes in die `.test`-Datei. Als Testmarken werden erkannt: `#DOK-xxxx`, `#SPEI-xxxx`, `#SPUR-xxxx`, `#PROT`, `#AC`, `#BZ`, `#STARTE`, `#TROMMEL` sowie die Geräteschalter `--leser1`, `--leser2`, `--stanzer1`, `--sw-ausgabe` und `--k0` (Bedeutung: `C8205-Interpreter-Durchführung_eines_Tests.md`, Abschnitte 2, 2b, 2e und 2g).

Die Zeilen dürfen an beliebiger Stelle des Quelltextes stehen, üblicherweise am Anfang. Davor erzeugt der Assembler nur die Zeile `Trommeldatei:`. Enthält der Quelltext keine einzige solche Zeile, erzeugt er stattdessen die Platzhalter `#DOK-0001`, `#SPEI-0001 0000-0000` und `#PROT STUFE=2`. Steht `#STARTE` im Vorspann, ersetzt es die Tastenfolge; der Assembler erzeugt dann keinen Abschnitt `Tastenfolge:`.

Andere Kommentarzeilen werden nicht übernommen, insbesondere nicht `--pruefe-…` (gehört in den Abschnitt `Prüfungen:`), `#INTERRUPT` (muss zwischen zwei Befehlszeilen der Trommeldatei stehen) und das Pragma `;#; LEITADRESSE` (3.3.12). Beispiel (im Quelltext):

```
;#DOK-0001 Druck von BZ und AC
; --sw-ausgabe virtuelle_Geraete/SW-Ausgabe
;#PROT STUFE=2
```

ergibt in der `.test`-Datei:

```
Trommeldatei:
#DOK-0001 Druck von BZ und AC
--sw-ausgabe virtuelle_Geraete/SW-Ausgabe
#PROT STUFE=2
0000: ... (die Zeilen aus dem Quelltext)
Tastenfolge:
H G R
```

Vor jedem Lauf löscht der Assembler die Dateien `.log`, `.obj`, `.mqo`, `.sym`, `.test` und `.dmp` des Projekts und erzeugt sie neu. Die früher genannten Endungen `.mqt` (modifizierter Quelltext) und `.l82` (Objektcode für den `C8205Simulator`) erzeugt der aktuelle Assembler nicht mehr. Dasselbe gilt für `.mac`: Der Quellcode legt keine solche Datei an. Mit `-o` bzw. `-t` (Abschnitt 2) wird statt der `.obj`- bzw. `.test`-Datei im Projektverzeichnis die dort angegebene Datei gelöscht und neu erzeugt; eine ältere Datei im Projektverzeichnis bleibt unberührt.

3.1a Schalter `--ABSPR` (absolut adressierter KEP-Lochstreifen)

Das KEP liest nur Dualziffern. Mit `--ABSPR` (gleichwertig `--ABSOLUTPROGRAMM` und `--ASBPR`, Groß-/Kleinschreibung egal) schreibt der Assembler deshalb statt der `.obj`-Datei die Datei `<name>.kep` (mit `-o` wie bei `.obj` umlenkbar). Sie ist ein Lochstreifen in der Form der Lochstreifen-Hilfsprogramme: je Lochungszeile 8 Ziffern 0/1 (Spurfolge `w v u p 8 4 2 1`), nur die Zeichen `0 1 : .` des R300-Codes in Kleinbuchstabenlage (ungerade Parität), also ohne weitere Umwandlung in den Leser 1 legbar.

Aufbau, aus den Zeilen, die sonst in die `.obj` kämen (die Kennziffern K1 entfallen, bestimmen aber die Aufbereitung):

Zeile	Ausgabe
Anfang	Leitbefehl `<start> 240:` mit der Adresse aus `start` (oktal, ohne Vornullen, höchstens 8 Oktaltriaden), dann der Operationscode 240
`00`	Rest der Zeile ohne Leerzeichen unverändert, mit `.` abgeschlossen
`01`	Adressstellen plus aktueller Wert `oo..oo` (Anfangswert = `start`), dann die 3 Oktalziffern des Operationscodes, ohne Vornullen, mit `.`
`20`	`oo..oo` wird durch den angegebenen Wert (ohne Leerzeichen) ersetzt und gilt für spätere Additionen; dazu ein neuer Leitbefehl `<wert> 240:`
`21`, Endezeichen `/`	entfallen

Jede Oktalziffer wird durch ihre 3 Bits ersetzt, führende Nullbits entfallen (Beispiele: `0040 240:` = `100000010100000:`, `0001 164.` = `1001110100.`). Das Pragma `LEITADRESSE` hat Vorrang und wird vor der Umsetzung ins Binäre angewendet: Ersetzt es 01 durch 00, wird K1=00 behandelt (Rest unverändert, keine Addition). Ersetzt es 01 durch 02 … 09, werden diese Zeilen nicht umgesetzt (sie fehlen in der `.kep`); der Assembler gibt dazu eine Warnung mit der Anzahl aus. Gibt es danach keine Zeile mit K1=01 mehr, gibt er eine Warnung aus; ist die Ursache das Pragma `LEITADRESSE 00` (absolut geschriebener Quelltext, der Regelfall beim Minimalprogramm), ist es nur ein Hinweis „Der Quelltext gilt als absolut adressiert, es wird keine Adresse umgerechnet“ (ab Version 1.00-030). Dasselbe gilt für `LEITADRESSE 00` zusammen mit `start <adresse>` ungleich 0: Der erste Leitbefehl lautet dann `<adresse> 240`, die Operanden bleiben aber unverändert (die Startadresse wird nicht addiert); der Assembler warnt (ab Version 1.00-028). Regel: Relativ geschriebener Quelltext mit `start <adresse>` ohne Pragma; absolut geschriebener Quelltext mit Pragma `LEITADRESSE 00` und `start` ohne Operand. Wird ein Wort auf eine Zelle von 0002 bis 0037 geladen (dort liegt das KEP selbst), gibt der Assembler eine Warnung mit der Zelle aus (ab Version 1.00-029; vorher warnte er bei jedem Leitbefehl auf 0000 bis 0037). Die Zellen 0000 und 0001 sind erlaubt und warnen nicht: Dort trägt das geladene Programm, wie das Minimalprogramm, seinen Sprung zur Startadresse ein; der Bahnsperrenschalter muss beim Laden aus sein. Das KEP selbst ist nicht Teil des Lochstreifens.

3.2 Sprachdefinition

```
<zahl>              ::= <oktalzahl> | <dezimalzahl>
<oktalzahl>          ::= oktalziffer==0 [ oktalziffer [ oktalziffer … ]]
<oktalziffer>        ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7
<vorzeichen>         ::= -
<dezimalziffer>      ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
<dezimalzahl>        ::= [<vorzeichen>] dezimalziffer<>0 [ dezimalziffer [ dezimalziffer … ]]
<alphazeichen>       ::= abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ
<sonderzeichen>      ::= % : + - / ` , . < " [ > ' ~ ] # * ( = ; ) ? !
<wert>               ::= <oktalzahl> | <dezimalzahl> | <symbol>
<rechenzeichen>      ::= + | - | * | /
<speicherplatz>      ::= Speicherstelle auf dem Trommelspeicher, fortlaufend nummeriert durch
                         eine Oktalzahl mit dem Wertebereich 00000 bis 07777.
<symbol>             ::= <adresssymbol> | <literalsymbol> | (leer)
<adresssymbol>       ::= Zeichenfolge aus 1 bis 32 Zeichen:
                         <alphazeichen> [ <alphazeichen> | <dezimalziffer> [ … ]],
                         abgeschlossen durch Doppelpunkt.
                         Sie wird vom Programmierer frei definiert. Der Unterstrich `_` wird ebenfalls akzeptiert.
                         Das erste Zeichen ist ein Buchstabe oder `_`, keine Ziffer. Erlaubt sind nur die
    ASCII-Buchstaben
                         `a`–`z`, `A`–`Z`, die Ziffern `0`–`9` und `_`; Zeichen wie `-`, `.` oder `ä` sind
    unzulässig.
                         Doppelt definierte Symbole, Symbole über 32 Zeichen und unzulässige Symbole sind Fehler.
<literalsymbol>      ::= 'l' (kleines L) gefolgt von einer Oktalziffernfolge von genau 4
                         Oktalziffern, abgeschlossen durch Doppelpunkt.
                         Sie wird vom Assembler automatisch aus dem Inhalt des Literalzählers vergeben.
<ausdruck>           ::= <werteausdruck> | <literal>
<operand>            ::= <werteausdruck> | <literal> | (leer)
<adressausdruck>     ::= <wert><rechenzeichen>[<wert><rechenzeichen> … ]
                         Einschränkungen: kein negatives Vorzeichen;
                         Wertebereich nur von 00000 … 07777.
                         Ein Vorzeichen am Anfang (auch `+`) ist nicht zulässig.
                         Verstöße meldet der Assembler als Fehler in Pass 3.
<werteausdruck>      ::= <wert><rechenzeichen>[<wert><rechenzeichen> … ]
                         Einschränkung: Bei `dw` und in `#`-Literalen ist ein relatives Ergebnis (eine einzelne
    symbolische
                         Adresse, auch mit Zahl) nicht zulässig. Zulässig: Zahlen, `abs`-Symbole, die paarweisen
                         Differenzen symbolischer Adressen (auch mehrere). Wertebereich -2^32 … 2^33-1.
                         Adressen werden mit `da` oder `@`-Literalen geschrieben.
<literal>            ::= #<werteausdruck> | @<adressausdruck>
                         Literal bedeutet, dass der numerische Wert von <werteausdruck> oder
                         <adressausdruck> auf einem vom Assembler selbst festgelegten
                         <speicherplatz> gespeichert wird. Diesem Speicherplatz wird vom
                         Assembler ein <literalsymbol> zugeordnet.
<adresse>            ::= numerischer Wert eines <symbol>, der einen <speicherplatz> adressiert.
<kommentar>          ::= ; (Semikolon) gefolgt von ASCII-Zeichen. | (leer)
                         Innerhalb der \-Zeichen von dt wird ; nicht als Kommentar betrachtet.
<blockkommentar>     ::= ```   als beginn auf einer Zeile für sich, ab Spalte 1
                         ```   als Ende auf einer Zeile für sich, ab Spalte 1
                         Dazwischen beliebiger Text
<trenner>            ::= Tabulator[<trenner>[...]] | Leertaste<trenner>[...]]
<befehl>             ::= [<symbol>] <trenner><operationscode> [<trenner><operand>]
                         [<trenner><kommentar>]
                         Alle Befehle müssen auf einer Zeile stehen; eine Zeile darf höchstens 255 Zeichen lang
    sein.
                         Ein <symbol> darf auch allein auf einer Zeile stehen (ohne <operationscode>). Die Zeile
    belegt keinen
                         Speicher; das Symbol erhält die Adresse der nächsten Zeile, die Speicher belegt, und der
    Befehlszähler
                         bleibt unverändert. Mehrere solche Symbole in Folge erhalten dieselbe Adresse. Vor start
    ist das ein
                         Fehler, wie ein Symbol auf der start-Zeile selbst.
<operationscode>     ::= <maschinenbefehl> | <assembleranweisung>
<assembleranweisung> ::= start | end | db | dbn | dw | da | dt | dto | dtm | dz | equ | equ10 | equ40 | equ100 |
    abs
<maschinenbefehl>    ::= <oktalziffer><oktalziffer><oktalziffer>
<quelltext>          ::= <befehl>[<befehl> [ … ]]
```

Auswertung von Ausdrücken (`<werteausdruck>` und `<adressausdruck>`): `*` und `/` binden stärker als `+` und `-`; Operatoren gleichen Ranges werden von links nach rechts ausgewertet (`2+3*4` ergibt 14, `10-2-3` ergibt 5). `/` ist eine ganzzahlige Division, deren Ergebnis zur Null hin abgeschnitten wird (`7/2` ergibt 3, `-7/2` ergibt -3); eine Division durch 0 ist ein Fehler. Ein Vorzeichen ist nur am Anfang des Ausdrucks zulässig (`-7/2`, `+5`, nicht `7/-2` oder `--5`). Klammern und Leerzeichen innerhalb eines Ausdrucks sind nicht zulässig.

Keine mnemonischen Operationscodes: Seit 1.00-026 kennt der Assembler nur noch dreistellige Oktalcodes (000–777). Die frühere Ansatztabelle mit `add`, `sub`, `ld`, `jmp`, `stopp` und `nop` (vorläufige Platzhalterwerte) ist entfallen; solche Namen ergeben jetzt den Fehler „Operationscode ist ungültig“.
```

3.3 Assembleranweisungen

3.3.1 Definiere Bereich

```
[<symbol>]  db  <zahl>
```

Hält <zahl> Speicherplätze frei, ohne ihren Inhalt zu definieren. <zahl> muss eine Oktalzahl (führende 0) oder eine Dezimalzahl sein, höchstens 010000. Negative Zahlen, Ausdrücke (z. B. 3+1), Symbole und ein fehlender Operand sind Fehler.

3.3.2 Definiere Bereich auf Null

```
[<symbol>]  dbn  <zahl>
```

Hält <zahl> Speicherplätze frei, setzt ihren Inhalt auf 0. Für <zahl> gilt dasselbe wie bei db. Je Speicherplatz erzeugt dbn in der Objektdatei die Zeile 00 0..

3.3.3 Definiere Wert

```
[<symbol>]  dw  <werteausdruck>
```

- Speichert den numerischen Wert von <werteausdruck> auf dem <speicherplatz>. Zulässig ist der Bereich -2^32 bis 2^33-1 (33-Bit-Wort), sonst meldet der Assembler Operand ist ungültig. Negative Werte werden im Zweierkomplement gespeichert (dw -5 ergibt 77777777773). Ein Ergebnis, das ein relativer Wert ist, ist im <werteausdruck> von dw nicht zulässig; das meint eine einzelne symbolische Adresse (dw a, dw a+1, Meldung Operand ist ungültig). Dagegen sind paarweise Differenzen symbolischer Adressen zulässig, auch mehrfach (dw ende-anfang, dw a-b+c-d). Auch absolute Symbole (abs) sind zulässig; Adressen schreibt man mit da.
- Es wird keine Hauptleitadresse addiert.
- In der Objektdatei steht ein Datenwort als 00 <elf Oktalziffern>.; nur der Wert 0 wird kurz als <adresse>: 0. ausgegeben (Nullwort ohne Kennzeichen).

3.3.4 Definiere Adresse

```
[<symbol>]  da  <adressausdruck>
```

Speichert den numerischen Wert von <adressausdruck> auf dem <speicherplatz>, verschoben um «9 in den Adressstellen. Zum <adressausdruck> wird die 1. Hauptleitadresse addiert.

Das Ergebnis von <adressausdruck> muss bei da (und bei @-Literalen) immer eine **relative Adresse** sein. Jedes relative Symbol (Marke, equ-Symbol, Literalsymbol) zählt dabei +1, wenn es addiert wird, und −1, wenn es subtrahiert wird; Zahlen und abs-Symbole zählen 0. Es muss mindestens ein relativer Adressanteil übrig bleiben (Summe ≥ 1). Unzulässig sind daher:

- Konstanten und konstante Ausdrücke (da 5, da absx, da absx+2),
- paarweise Differenzen relativer Adressen, die sich aufheben (da a-b, da b-a+absx),
- relative Adressen in * oder / (da a*2).

Zulässig sind z. B. da a, da a+5, da a-b+c, da a+absx und da *. Verstöße meldet der Assembler in Pass 2 als Fehler mit Datei und Zeile (Operand: "a-b" ist bei da nicht zulässig: …). Konstante Werte schreibt man mit dw.

```
[<symbol>]  da  <literal>
```

- Erzeugt am Programmende automatisch aus <literal> eine Definition der Art <literalsymbol> da <adressausdruck>.
- Zum <adressausdruck> wird die 1. Hauptleitadresse addiert.
- Nimmt stattdessen die <adresse> von <literalsymbol> in den Befehl, der das <literal> enthält.

3.3.5 Definiere Text zur Ausgabe von Zeichen

```
[<symbol>]  dt[o|m]  \<alphazeichen> | <leerzeichen> | <dezimalziffer>
             <sonderzeichen> [ <alphazeichen> | <leerzeichen> | <dezimalziffer> | <sonderzeichen>[...]]\
```

- Gespeichert wird ein Code gemäß R300-Code Ausgabeadresse C8205 (siehe R300-code.pdf). Die Abspeicherung erfolgt als Folge von Speicherplatzinhalten in der Form (oktal): `0000 xxxx 030`, wobei `xxxx` die Ausgabeadresse ist.
- Die Zeichen „\" am Anfang und Ende sind erforderlich, wenn der Text mit `#` beginnt, ein `;` enthält (sonst beginnt dort der Kommentar) oder führende bzw. abschließende Leerzeichen enthalten soll. Leerzeichen innerhalb des Textes (`dt a b`) sind auch ohne „\" zulässig; sonst werden führende und abschließende Leerzeichen entfernt. Ein führendes `#` kennzeichnet ein Literal (`#<werteausdruck>`), keinen Text; `dt #ab` und `dto #x` meldet der Assembler als Fehler. Richtig ist `dt \#ab\` bzw. `dto \#x\`. Ein `#` innerhalb des Textes (`dt a#b`) ist zulässig.
- `dt` ist für längere, zusammenhängende Textausgaben gedacht. Sein Vorteil: Der Text lässt sich ohne ausgeschriebene Umschaltzeichen definieren, der Quelltext bleibt gut lesbar. Für verstreute Einzelzeichen ist `dt` nicht geeignet, dafür nimmt man `dz` (3.3.6) und schreibt alle Umschaltzeichen explizit.
- Der Assembler setzt `dt` zur Assemblerzeit in Maschinenbefehle um und kennt den Zustand KB/GB zur Laufzeit nicht. Er führt den Zustand nur in der Reihenfolge des Quelltexts weiter. Beim allerersten `dt` des Quelltexts nimmt er KB an. Jedes weitere `dt` setzt den Zustand des vorangehenden `dt` fort und fügt nur dort ein Umschaltzeichen ein, wo sich der Zustand innerhalb des Textes ändert.
- Daraus folgt: Vor jeder zusammengehörenden Folge von `dt`-Anweisungen muss ein Umschaltbefehl durchlaufen werden, der exakt auf den Zustand des ersten Zeichens dieser Folge schaltet (`dz $inkb`, `dz $ingb` oder `dtm` als erste Anweisung der Folge). Das gilt für jede Folge neu. Es kann dabei ein eigentlich unnötiger Umschaltbefehl entstehen. Der ist beabsichtigt, weil sich Assemblerzeit und Laufzeit nicht synchronisieren lassen. Umschaltzeichen mit `dz` zwischen zusammengehörenden `dt`-Anweisungen sind nicht zulässig, sie bringen den Zustand des Assemblers durcheinander.
- Praktischer Hinweis dazu: `dtm` fügt immer vor dem ersten Zeichen ein Umschaltzeichen passend zum ersten Buchstaben ein. Damit ist es die natürliche erste Anweisung einer `dt`-Folge. Es ersetzt das von Hand gesetzte `dz $inkb` oder `dz $ingb` davor.
- `dto` verhält sich wie `dt`, außer dass generell KEINE Umschaltzeichen automatisch eingefügt werden. Das bedeutet, in einer `dto`-Anweisung kann man KEINE gemischte Groß- und Kleinschreibung definieren (auch keine Zahlen und Großbuchstaben usw.).
- `dtm` verhält sich wie `dt`, außer dass vor dem ersten Zeichen generell ein Umschaltzeichen, dem ersten Buchstaben entsprechend, automatisch eingefügt wird.

3.3.6 Definiere Zeichen (spezielle Sonderzeichen)

```
[<symbol>]  dz  $wrzl | $sm | $inkb | $ingb | $irrz | $irrs | $trans | $bm | $vorl |
                $irrb | $wr | $tab | $zr | % | : | + | ` | / | , | . | -
```

- Gespeichert wird ein Code gemäß R300-Code Ausgabeadresse C8205 (siehe Tabelle 2 unten). Die Abspeicherung erfolgt auf einem Speicherplatz in der Form (oktal): `0000 ssss 030`. Eine automatische Ergänzung von Umschaltzeichen wie bei `dt` erfolgt hier generell NICHT.
- Findet man z. B. `dz $wrzl` zu kompliziert, kann man das mit einem Makro vereinfachen:

  ```
  WRZL:  MACRO
             dz  $wrzl  ; Zeilenschaltung
         MEND
  ```

  (In der mitgelieferten `c82ass.makros` enthält `WRZL` zusätzlich die Anweisung `ANWAHL 4700`.)
- Das Zeichen `;` kann bei `dz` nicht angegeben werden, weil es einen Kommentar einleitet; der Operand für den Code 0150 ist `:` (vgl. Tabelle 2 und 3).

3.3.7 Beginn des Quellcodes

```
start  [<zahl>]
```

- Nur als 1. Anweisung im `<quelltext>` zulässig, und nur einmal; ein Label vor `start` ist ein Fehler. Der Operand muss eine Konstante sein. Der Befehlszähler beginnt unabhängig vom Operanden bei 0. Dasselbe gilt für ein Label allein auf einer Zeile vor `start`. Eine Maschinencodeausgabe erfolgt nicht; mit Operand wird im Objektcode die Kennzeichenzeile `21 0000 <zahl>` ausgegeben.
- Ist `<zahl>` angegeben, wird die 1. Leitadresse in den „Lochstreifen" ausgegeben. Es wird dann durch `c82load` auf diese 1. Leitadresse + Ladeadresse geladen. Die Ladeadresse existiert dabei nur beim Laden mit `c82load`.

- Das Kennzeichen 01, das für die 1. Leitadresse steht, kann mit dem Pragma `;#; LEITADRESSE xx` durch ein anderes Kennzeichen ersetzt werden (Abschnitt 3.3.12). Das Pragma muss vor `start` stehen.
- Mit `;#; LEITADRESSE 00` wird dem Objektcode kein Leitbefehl vorangestellt: Die Kennzeichenzeile `21 0000 <zahl>` entfällt (im Objektcode und im Listing); der Operand von `start` hat dann keine Wirkung auf Objektcode und Listing (Abschnitt 3.3.12).

3.3.8 Definition Relativadresse

```
[<symbol>]  equ  <werteausdruck> | *
```

- Definiert für das `<symbol>` einen `<werteausdruck>`. Der Befehlszähler wird auf diesen Wert gesetzt.
- Ist `*` angegeben, wird dem Symbol stattdessen der aktuelle Befehlszählerstand zugewiesen.
- Eine Maschinencodeausgabe erfolgt nicht. Im Objektcode (Eingabe mit Kennzeichen, Format des Minimalprogramms) wird der Kennzeichenbefehl für das Setzen der Relativadresse ausgegeben (KZ=20).
- Das weitere Laden erfolgt auf 1. Leitadresse + Ladeadresse + Relativadresse. Die Ladeadresse existiert dabei nur beim Laden des Objektcodes.
- Die Anweisungen `equ100`, `equ40` und `equ10` runden den Befehlszähler (Operand `*`) oder den numerischen Operanden auf das nächste Vielfache von 0100, 040 bzw. 010 (oktal) auf.

3.3.9 Definition Absolutadresse

```
[<symbol>]  abs  <oktalzahl> | <dezimalzahl>
```

- Definiert für das `<symbol>` eine absolute `<adresse>`. Weder Objektcode- noch Maschinencodeausgabe erfolgt.
- Beachte: Symbole, die durch `abs` definiert wurden, sind nicht verschieblich. Anweisungen, in denen ein solches Symbol im Adressausdruck oder Werteausdruck vorkommt, werden mit K1K2=00 generiert!
- `abs` kann mehrfach mit dem gleichen Wert definiert werden, ohne Fehler zu erzeugen, aber nicht mehrfach mit dem gleichen Symbol für unterschiedliche Werte.

3.3.10 Definition Absolute Sektoradresse

```
[<symbol>]  spa  <oktalzahl> | <dezimalzahl>      im Bereich 00..77 (oktal)
```

- Dieser Befehl verhält sich wie ein `equ *`, nur dass die rechten beiden Oktalziffern durch den Wert von `<oktalzahl> | <dezimalzahl>`, dargestellt in 2 Oktalziffern, ersetzt werden.
- Beispiel: `spa 020` macht, wenn die Anweisung auf `*` steht, mit dem Beispielwert `*==0256` das Gleiche wie `equ 0220`. Steht `spa 020` auf 0277, wirkt sie wie `equ 0220`. Steht `spa` auf 0301, wirkt sie wie 0320.

3.3.11 Ende des verschieblichen Teils des Quelltextes

```
[<symbol>]  end
```

- Hinter dieser Anweisung sollten nur noch `abs`- oder `equ`-Anweisungen folgen. Sie werden auch dann ausgewertet, wenn das Programm Literale enthält. Makrodefinitionen (siehe Abschnitt 4.1) dürfen ebenfalls dort stehen. Der Assembler ordnet hinter dem ersten „end" die automatisch erzeugten Literaldefinitionen ein, sodass sie noch zum verschieblichen Teil hinzukommen. Ein nachfolgendes `equ` käme also HINTER die Literale. Ein weiteres „end" ändert daran nichts.
- Eine Maschinencodeausgabe erfolgt für „end" nicht.
- Code und Daten (Maschinenbefehle, `db`, `dw` usw.) hinter „end" sind nicht vorgesehen. Der Assembler lehnt sie nicht ab, sondern übersetzt sie hinter den Literalen (soweit vorhanden) mit.

3.3.12 Pragma LEITADRESSE

```
;#; LEITADRESSE <xx>
```

- Ein Pragma ist eine Kommentarzeile, die mit `;#;` beginnt und dem Assembler eine Anweisung gibt. Es ist keine Assembleranweisung im Sinne der Abschnitte 3.3.1 bis 3.3.11, erzeugt weder Maschinencode noch ein Wort auf der Trommel und steht auf einer eigenen Zeile (Einrückung mit Leerzeichen oder Tabulator ist zulässig). Ein anderes Kommentar, das mit `;#;` beginnt, bleibt ein gewöhnliches Kommentar.

- `<xx>` ist genau zweistellig und liegt zwischen 00 und 09 (`02`, nicht `2`; `10` ist ein Fehler). Danach folgt nur noch Leerraum oder ein Kommentar (`;`). Das Schlüsselwort `LEITADRESSE` wird ohne Beachtung der Groß-/Kleinschreibung erkannt und vom Wert durch Leerraum getrennt.
- Das Pragma darf höchstens einmal vorkommen und muss **vor** `start` stehen. Ohne Pragma gilt 01.
- Wirkung: Im Objektcode (`.obj`) und im Listing (STDOUT und `.dmp`) steht anstelle des Kennzeichens 01 (K1K2=01, bezogen auf die 1. Leitadresse, siehe 3.3.7 und 3.3.8) das Kennzeichen `<xx>`. Das betrifft alle Zeilen, die sonst mit 01 beginnen. Zeilen mit anderen Kennzeichen bleiben unverändert. Die Dateien `.test`, `.mqo`, `.sym` und `.log` enthalten kein Kennzeichen und ändern sich nicht.
- Sonderfall `;#; LEITADRESSE 00`: Der Code wird so als Objektcode (`.obj`) ausgegeben, wie ihn der Assembler erzeugt hat, aber mit dem Kennzeichen 00 statt 01. Das dient dazu, bereits absolut codierte Programme im Objektcode-Format auszugeben (K1K2=00: keine Adressänderung beim Laden, vgl. 3.3.9). Dem Objektcode wird dabei kein Leitbefehl vorangestellt: Die Kennzeichenzeile `21 0000 <zahl>`, die `start` mit Operand sonst erzeugt (3.3.7), wird weder in die `.obj` noch in das Listing (STDOUT und `.dmp`) geschrieben. Die Kennzeichenzeilen `20 0000 <adresse>` für `equ` bleiben erhalten. Im Objektcode sind die früheren 01-Wörter dann von den übrigen 00-Wörtern nicht mehr zu unterscheiden.
- Hat der Quelltext das Pragma, gibt der Assembler auf STDERR die Statuszeile `Leitadresse: <xx>` aus (nach `Benutze für -s die Datei:`). Das gilt auch für `;#; LEITADRESSE 01`.
- Fehler (jeweils `Fehler: Zeile <n> der Quelldatei: …`, danach `Fehler: Abbruch.`, Rückgabecode 16, keine Ausgabedateien):
  - Das Pragma steht hinter `start`: „Das Pragma LEITADRESSE muss vor `start` stehen.“
  - Das Pragma kommt ein zweites Mal vor: „Das Pragma LEITADRESSE darf nur einmal vorkommen.“
  - Der Wert fehlt, ist nicht zweistellig oder liegt außerhalb von 00 … 09: „Das Pragma LEITADRESSE erwartet einen zweistelligen Wert von 00 bis 09 (z. B. ;#; LEITADRESSE 02).“
- Achtung: Ein falsch geschriebenes Schlüsselwort (zum Beispiel `LEITADRESS`) oder ein Wert ohne Trennung vom Schlüsselwort (`LEITADRESSE02`) wird nicht gemeldet. Die Zeile gilt dann als gewöhnliches Kommentar, und das Kennzeichen bleibt 01. Ob das Pragma gewirkt hat, zeigt die Statuszeile `Leitadresse:`.

Beispiel (`Projekt_1/bsp_5.16/bsp_5_16.asm`, Quelltext beginnt mit der Zeile `;#; LEITADRESSE 03`):

Quelltextzeile	`.obj` **ohne Pragma**	`.obj` **mit** `;#; LEITADRESSE 03`
`000  go`	`01 2000 000.`	`03 2000 000.`
`equ  02000`	`20 0000 2000.`	`20 0000 2000.`
`aqv: da  qvektor`	`01 2027000.`	`03 2027000.`
`dqv: dw  05000`	`00 00000005000.`	`00 00000005000.`
`i: db  1`	`20 2007.`	`20 2007.`
`164  0001`	`01 0001 164.`	`03 0001 164.`

Im Listing (`.dmp` und STDOUT) steht in denselben Zeilen entsprechend `03` statt `01` in der Kennzeichenspalte, zum Beispiel `0000: 03 2000 000.`

Beispiel für `00` (kleines Programm, das nicht zum Projekt gehört; der Quelltext beginnt mit der Zeile `;#; LEITADRESSE 00`, danach folgt `start 0100`):

Quelltextzeile	`.obj` **ohne Pragma**	`.obj` **mit** `;#; LEITADRESSE 00`
`start 0100`	`21 0000 0100.`	keine Zeile
`anf: 211 wert`	`01 0020 211.`	`00 0020 211.`
`240 ziel`	`01 0021 240.`	`00 0021 240.`
`000 anf`	`01 0000 000.`	`00 0000 000.`
`equ 020`	`20 0000 0020.`	`20 0000 0020.`
`wert: dw 5`	`00 00000000005.`	`00 00000000005.`
`ziel: da anf`	`01 0000.`	`00 0000.`
`020 0`	`01 0000 020.`	`00 0000 020.`

Im Listing (`.dmp` und STDOUT) steht entsprechend `00` statt `01` (zum Beispiel `0000: 00 0020 211`), und die Zeile `21 0000 0100` fehlt.

Tabelle 2: Sonderzeichen für die dz-Anweisung

Spalte 1: String, der als Operand bei `dz` anzugeben ist. Spalte 2: 4 Oktalziffern, die die 8 Bit des 030-Ausgabecodes enthalten.

Sonderzeichen-String	Oktalcode (030-Ausgabe)
`$wrzl`	02720
`$sm`	02720
`$inkb`	01170
`$ingb`	0570
`$irrz`	03770
`$irrs`	0160
`$trans`	01770
`$bm`	03520
`$vorl`	0
`$irrb`	0760
`$zr`	0320
`$tab`	01550
`$wr`	01120
`%`	01740
`:`	0150
`+`	0400
`` ` ``	01600
`/`	01410
`,`	01530
`.`	0730
`-`	01000

Tabelle 3: Ausgabe-Umschlüsselung außer Sonderzeichen dz

Spalten: ASCII-Kleinbuchstabe, ASCII-Großbuchstabe, Oktalcode (8 Bit des 030-Ausgabecodes), ASCII-Code Großbuchstabe (dez.), ASCII-Code Kleinbuchstabe (dez.).

Klein	Groß	Oktalcode	ASCII Groß (dez)	ASCII Klein (dez)
1	<	0010	60	49
2	"	0020	34	50
3	≈	0230	0	51
4	[	0040	91	52
5	>	0250	62	53
6	'	0260	39	54
7	≋	0070	0	55
8	~	0100	126	56
9	]	0310	93	57
0	#	0200	35	48
%	*	1740	42	37
Tabulator	Tabulator	1550	9	9

Klein	Groß	Oktalcode	ASCII Groß (dez)	ASCII Klein (dez)
q	Q	1300	81	113
w	W	1660	87	119
e	E	0450	69	101
r	R	1110	82	114
t	T	1630	84	116
z	Z	1710	90	122
u	U	1440	85	117
i	I	0510	73	105
o	O	1060	79	111
p	P	1270	80	112
:	(	0150	40	58
+	=	0400	61	43
a	A	0610	65	97
s	S	1420	83	115
d	D	0640	68	100
f	F	0460	70	102
g	G	0670	71	103
h	H	0700	72	104
j	J	1210	74	106
k	K	1220	75	107
l	L	1030	76	108
` | ; | 1600 | 59 | 96				
/	)	1410	41	47
SatzMarke	WRZL	2720	10	10
y	Y	1500	89	121
x	X	1470	88	120
c	C	0430	67	99
v	V	1650	86	118
b	B	0620	66	98
n	N	1050	78	110
m	M	1240	77	109
,	?	1530	63	44
.	!	0730	33	46
-	—	1000	—	45
Leertaste	ZwR	0320	32	32

(Zeile "-" hat in der Quelle keine Einträge in den Spalten "Groß" und "ASCII Groß (dez)" — als "—" markiert, nicht ergänzt.)

4. Der Makro-Präprozessor des Assemblers

Der Präprozessor ist sehr einfach gehalten und hat eine starre Syntax. Er prüft, ob eine Operation ein Makroname ist. Die Prüfung erfolgt in der Textdatei `c82ass.makros`, die erweitert werden darf. Der Assembler sucht sie unter diesem festen Namen zuerst im Arbeitsverzeichnis des Aufrufs, sonst unter `Sourcecode/c82ass.makros`; fehlt sie an beiden Orten, bricht er mit einer Fehlermeldung ab. Zusätzlich können Makros direkt im Quelltext definiert werden (Abschnitt 4.1).

- Jedes dort definierte Makro beginnt mit dem Makronamen, unmittelbar gefolgt von einem Doppelpunkt und danach (getrennt durch Tab oder Leerzeichen) dem Schlüsselwort `MACRO`.
- Müssen dem Makro Parameter übergeben werden, folgt nach `MACRO`, getrennt durch Leerzeichen, die Anzahl der Parameter. Es sind maximal 9 Parameter zulässig.
- Danach kommen die Definitionsanweisungen des Makros. Sie können die formalen Parameter `$x` mit x=1..9 enthalten, die durch die Übergabeparameter ggf. mehrfach ersetzt werden.
- Jedes Makro wird durch das Schlüsselwort `MEND` abgeschlossen (Leerraum davor ist beliebig).

Im Definitionsteil kann für jeden Parameter eine Prüfliste angegeben werden. Ist sie angegeben, darf der betreffende Parameter nur die Werte der Liste annehmen. Eine Prüfliste wird durch `$PRx` mit x=1..9 definiert; sie muss auf einer Zeile für sich stehen.

Statt einer Werteliste kann hinter `$PRx` eine Typangabe stehen (z. B. `$PR1: $TYPADR`). Dann prüft der Assembler beim Makroaufruf, ob der übergebene Wert zum Typ passt; sonst meldet er einen Fehler `Parameter $x: Wert "<wert>" …`:

Typ	Zulässiger Wert beim Aufruf
`$TYPADR`	eine symbolische Adresse (ein Label, z. B. `marke`) oder eine Sternadresse (`*`, `*+3`); keine Zahl
`$TYPKON`	eine Oktalkonstante: nur Ziffern 0–7, beginnt mit `0` (z. B. `0100`); `100` oder `09` sind Fehler
`$TYPABS`	ein Symbol, das im Quelltext mit `abs` definiert ist (z. B. `absx: abs 05`)

Die Prüfung erfolgt vor der eigentlichen Übersetzung und ist deshalb nur syntaktisch: Bei `$TYPADR` wird nicht geprüft, ob das Label im Programm vorkommt (das meldet die spätere Symbolauflösung).

Die Definition der zu generierenden Anweisungen kann durch eine `$IF`-`$ELIF`-`$ENDIF`-Struktur gesteuert werden. Sie kann mehrfach auftreten, darf aber nicht geschachtelt werden. Die Schlüsselworte `$IF`, `$ELIF`, `$ENDIF` müssen jeweils auf einer Zeile für sich stehen.

Hinter `$IF` und `$ELIF` muss mindestens einer, können bis zu 4 Bedingungsblöcke angegeben werden. Ein Bedingungsblock besteht aus 3 Strings: `<wert1> <vergleichszeichen> <wert2>`. Als `<vergleichszeichen>` sind >, <, =, >=, <= zugelassen. Jeder folgende Bedingungsblock ist durch `|` (logisch ODER) oder `&` (logisch UND) zu verknüpfen.

Achtung: Die Bedingungsblöcke haben eine implizite Klammerung der Art:

```
(((  <bedingungsblock1> [|&] <bedingungsblock2> ) [|&] <bedingungsblock3> ) [|&] <bedingungsblock4> )
```

Innerhalb eines Definitionsblockes sind Kommentare zugelassen: `#` leitet einen Kommentar ein, der nur in der Makrodefinition gilt und auf einer eigenen Zeile stehen muss; `;` leitet einen Kommentar ein, der auch generiert wird — er kann auf einer eigenen Zeile oder in der Anweisung vorkommen.

Beispiel:

```
ANWAHL:  MACRO 1
  $PR1 4000 4100 4200 4300 4400 4500 4600 4700 K0 K1 K2 K3 K4 K5 K6 K7 LSL1 LSL2 LKL1 LKS1 LSS1 SW1I SW1O SW2O KONI
  # Generieren einer Anwahl für Zeichenkanäle
      $IF  $1  =  4000 | $1 = K7
         030 04000  ; 4000 oder K7
      $ELIF  $1  =  4100 | $1 = K6
         030 04100  ; 4100 oder K6
      $ELIF  $1  =  4200 | $1 = K5
         030 04200  ; 4200 oder K5
      $ELIF  $1  =  4300 | $1 = K4 | $1 = LKS1
         030 04300  ; 4300 oder K4 oder LKS1
      $ELIF  $1  =  4400 | $1 = K3 | $1 = LKL1 | $1 = SW2O
         030 04400  ; 4400 oder K3 oder LKL1 oder SW2 Output
      $ELIF  $1  =  4500 | $1 = K2 | $1 = LSL2 | $1 = SW1O
         030 04500  ; 4500 oder K2 oder LSL2 oder SW1 Output
      $ELIF  $1  =  4600 | $1 = K1 | $1 = LSL1 | $1 = LSS1
         030 04600  ; 4600 oder K1 oder LSL1 oder LSS1
      $ELIF  $1  =  4700 | $1 = K0 | $1 = SW1I | $1 = KONI
         030 04700  ; 4700 oder K0 oder SW1 Input oder KON Input
      $ENDIF
         MEND
```

Aufruf: `ANWAHL LSS1` erzeugt: `030  04600  ; 4600 oder K1 oder LSL1 oder LSS1`

Der Makroaufruf im Assemblerprogramm darf ein Label haben, z. B.:

```
input_loop:  ANWAHL  LSL1
             …
```

In einem solchen Fall wird generiert:

```
input_loop:  030  04600  ; 4600 oder K1 oder LSL1 oder LSS1
```

Der Makroaufruf selbst wird nicht als Kommentarzeile (`;  ANWAHL  LSL1`) ausgegeben; er erscheint weder im Listing noch in `.mqo`, `.dmp` oder `.test`.

Enthält das Makro selbst eine Schleife mit einem Label, z. B.:

```
BSP1:  MACRO
# Makro enthält eine Schleife
ML0001%: 211  …
         …
         000  ML0001%
         MEND
```

so wird bei einem Makroaufruf `loop: BSP1` folgende Anweisungsfolge generiert, wenn die erste zu generierende Anweisung ein eigenes Label hat:

```
loop      equ  *
ML0001DKR 211  …
          …
          000  ML0001DKR
```

Hat die erste Anweisung kein eigenes Label, entfällt die `equ`-Anweisung und die erste Anweisung erhält das Label des Makroaufrufs. Ausnahme: Ist die erste Anweisung `equ`, `abs`, `start` oder `end`, bleibt die `equ`-Anweisung bestehen.

Ein Makro SOLLTE kein „normales" Label der Assemblersprache enthalten, sonst kann es nur einmalig aufgerufen werden. Stattdessen muss ein generisches Label verwendet werden: genau `ML`, vier Ziffern und `%`, danach der Doppelpunkt (z. B. `ML0004%:`). Jedes andere Label mit `%` meldet den Fehler `Macrolabel muß mit ML beginnen …`. Bei jedem Aufruf wird das `%` durch drei Großbuchstaben ersetzt (z. B. `ML0001DKR`), sodass jeder Aufruf ein eigenes Label erhält.

Makro-Aufrufparameter, die exakt 4 Ziffern enthalten und nur diese, werden ggf. als 4-Ziffern-String geprüft (durch `$PRx`); sie erhalten bei der Benutzung (Ersetzung des formalen Parameters) automatisch eine Vornull. Beispiel: `ANWAHL 4600` erzeugt:

```
     030    04600  ; 4600 oder K1 oder LSL1 oder LSS1
```

4.1 Makros im Quelltext

Makros können nicht nur in der Makrodatei `c82ass.makros`, sondern auch direkt im Quelltext (`.asm`) definiert werden. Die Definition hat dieselbe Syntax wie in der Makrodatei: Sie beginnt mit dem Makronamen, unmittelbar gefolgt von einem Doppelpunkt, danach folgt das Schlüsselwort `MACRO` (bei Bedarf mit der Anzahl der Parameter). Abgeschlossen wird sie durch `MEND` (zulässig sind auch `MACROENDE` und `macroende`). Der Doppelpunkt hinter dem Makronamen ist im Quelltext Pflicht.

- **Gültigkeit:** Eine Definition darf an beliebiger Stelle im Quelltext stehen, auch vor `start`. Das Makro gilt im gesamten Quelltext, auch für Aufrufe, die vor der Definition stehen. Die Definition selbst erzeugt keinen Code.
- **Definitionsteil:** Es gelten alle Regeln dieses Abschnitts (formale Parameter `$x`, Prüflisten `$PRx`, `$IF-$ELIF-$ENDIF`, Kommentare mit `#` und `;`, generische Labels mit `%`). Ein Makro darf andere Makros aufrufen, gleichgültig ob sie in der Makrodatei oder im Quelltext definiert sind, aber nicht sich selbst.
- **Gleicher Name wie in der Makrodatei:** Das Makro im Quelltext ersetzt das gleichnamige Makro der Makrodatei, jedoch nur für diesen Assemblerlauf; die Makrodatei selbst bleibt unverändert. Der Assembler gibt dazu eine Warnung aus. Die Ersetzung wirkt auch dort, wo ein anderes Makro das ersetzte Makro aufruft.
- **Fehler:** Ein Makroname darf im Quelltext nur einmal definiert werden. Weitere Fehler sind eine `MACRO`-Zeile ohne Namen, eine Definition innerhalb einer anderen Definition (Schachtelung ist nicht erlaubt), ein fehlendes `MEND` und ein `MEND` ohne zugehöriges `MACRO`. Der Assembler meldet den Fehler und bricht nach Pass 0 ab.

Beispiel:

```
ZEILE2:  MACRO
         dz  $wrzl  ; Zeilenschaltung
         dz  $wrzl  ; Zeilenschaltung
         MEND
```

Der Aufruf `ZEILE2` erzeugt die beiden Anweisungen `dz $wrzl`.

Soll das Makro `WRZL` der Makrodatei nur für ein Programm anders arbeiten, wird es im Quelltext neu definiert:

```
WRZL:    MACRO
         dz  $wrzl  ; Zeilenschaltung
         dz  $wrzl  ; Zeilenschaltung
         MEND
```

Der Assembler meldet dazu:

```
Warnung: Makro WRZL aus der Makrodatei wird für diesen Lauf durch die Definition im Quelltext ersetzt.
```

5. Compilierungsbeispiel

```
c82ass.sh -p Projekt_1/bsp_5.16 -s bsp_5_16
```

5.1 Quelltext bsp_5_16.asm und Assemblerlauf

Hinweis: Die Datei `Projekt_1/bsp_5.16/bsp_5_16.asm` ist eine veränderte Fassung des hier abgedruckten Beispiels: Sie beginnt mit dem Startsprung `000 go` mit `equ 02000` und der Marke `go:`, enthält vor dem Halt den Debug-Befehl `164 0001`, und der Datenbereich beginnt mit `equ 02100` statt `equ 0100`. Ihr Objektcode weicht deshalb von 5.2 ab; der abgedruckte Quelltext ergibt genau den Objektcode in 5.2.

```
Projektverzeichnis: Projekt_1/bsp_5.16
Benutze für -s die Datei: bsp_5_16
Startadresse: 0000
C8205 Assembler C82aas Version 1.00

Pass 0
Verarbeite 'c82ass.makros', Erzeuge Macroersetzungen

Pass 1
Löse Literale auf

Pass 2
Löse Symbole auf nach Projekt_1/bsp_5.16/bsp_5_16.log und Projekt_1/bsp_5.16/bsp_5_16.sym

Pass 3
Erzeuge Kennzeichen-Befehle in Objektdatei Projekt_1/bsp_5.16/bsp_5_16.obj
```

Quelltext (`; C8205 Heft 2 Programmierung in Maschinensprache; Beispiel: 5.16 Umspeichern eines Vektors`):

```
                start
                000     beginn
aqv:            da      qvektor    ; Adresse der ersten Komponente Quellvektor
dqv:            dw      05000      ; Dimension von qvektor gezählt in Adresseinheiten
azv:            da      zvektor    ; Adresse der ersten Komponente Zielvektor
adreinheit:     dw      01000      ; Adresseinheit
spbef:          dw      0240       ; Speicherbefehl
i:              db      1          ; Arbeitszelle für i
beginn:         241     i          ; I:=0
schleife:       210     azv        ; ac:=(azv+i)
                210     spbef      ; ac:=(azv+i)240
                240     genbef     ; Arbeitszelle genbef:= (qzv+i) 240
                211     i          ; ac:=<i>
                210     aqv        ; ac:=qvektor+i
; 251: Laden des Inhaltes einer Speicherstelle, deren Adresse im AC steht, in den AC.
                251     0          ; ac:=<qvektor+i>
genbef:         db      1          ; hier steht der Befehl 240 (qzv+i) Speichern
                211     i          ; ac:=<i>
                210     adreinheit ; ac:=i+1
                240     i          ; i:=ac
                330     dqv        ; vergleicher 3, falls i == <dqv>
                170     halt       ; sprung bei gleich
                000     schleife
halt:           020     0          ; halt
qvektor:        dw      1
                dw      2
                dw      3
                dw      4
                dw      5
                equ     0100
zvektor:        db      5
ende:           end
```

Nach dem Listing meldet der Assembler:

```
Ausgabe als Minimalprogramm-Objektdatei nach: Projekt_1/bsp_5.16/bsp_5_16.obj
Ausgabe der formatierten Speicherbelegung nach: Projekt_1/bsp_5.16/bsp_5_16.dmp
Ausgabe der Testdatei (Spalten 1,3,4,5 fuer K1=00/01) nach: Projekt_1/bsp_5.16/bsp_5_16.test
```

5.2 Durch c82ass erzeugter Objektcode (K1K2-Programmcode) für die Eingabe mit Kennzeichen (Format des Minimalprogramms)

```
0000: 01 0007 000.
0001: 01 26000.
0002: 00 00000005000.
0003: 01 100000.
0004: 00 00000001000.
0005: 00 00000000240.
0006: 20 0007.
0007: 01 0006 241.
0010: 01 0003 210.
0011: 01 0005 210.
0012: 01 0016 240.
0013: 01 0006 211.
0014: 01 0001 210.
0015: 00 0000 251.
0016: 20 0017.
0017: 01 0006 211.
0020: 01 0004 210.
0021: 01 0006 240.
0022: 01 0002 330.
0023: 01 0025 170.
0024: 01 0010 000.
0025: 01 0000 020.
0026: 00 00000000001.
0027: 00 00000000002.
0030: 00 00000000003.
0031: 00 00000000004.
0032: 00 00000000005.
      20 0000 0100.
0100: 20 0105.
```

6. Abgleich mit dem Assembler c82ass

Dieser Abschnitt stammt nicht aus der .odt. Er hält fest, was am Verhalten des Assemblers `c82ass` auffällt und in der .odt nicht beschrieben ist, und wie der Quelltext `ca82ass.c` an die erneuerte .odt angepasst wurde. Grundlage ist der Quelltext (Stand 16.10.2026); ein Lauf mit neu erzeugtem Binary steht noch aus.

6.1 Auffälligkeiten im Assembler

Nr.	Stelle	c82ass tut	Anmerkung
1	`dw <literal>` (3.3.3)	`dw #5` und `dw @a` weist Pass 1 mit einer Fehlermeldung ab („Operand … ist bei dw nicht zulaessig: ein Literal ist eine relative Adresse“). Das Linux-Binary vom 15.10.2026 hatte `dw #5` noch ohne Meldung angenommen und die Adresse der Literalzelle statt des Werts eingesetzt; im Quelltext ist das behoben.	Die .odt beschreibt `dw <literal>` nicht. Den Wert 5 erhält man mit `dw 5`, Adressen mit `da`. Offen: Binary neu erzeugen und das Verhalten bestätigen.

6.2 Angleichung an die erneuerte .odt (16.10.2026)

Nr.	Stelle	Änderung in c82ass	Anmerkung
1	`<adressausdruck>` (3.2) bei `da` und `@`-Literalen	Die Prüfung „höchstens ein Symbol“ entfällt. Ausdrücke mit mehreren Symbolen, z. B. `da marke+absx`, werden angenommen.	Die Auswertung ist unverändert (Summe bzw. Differenz der Werte; K1K2=00, sobald ein `abs`-Symbol vorkommt, sonst 01, vgl. 3.3.9). Die Differenz zweier Marken (`da a-b`) wurde damals mit der Leitadresse verschoben; entschieden am 21.10.2026: `da a-b` ist unzulässig, siehe 6.14.
2	`<werteausdruck>` bei `dw` (3.2, 3.3.3)	Mehrere Differenzen (`dw a-b+c-d`) konnte der Assembler schon auswerten. Geändert sind nur Kommentare und der Fehlertext: „erlaubt sind nur paarweise Differenzen von Adressen“.	–

6.3 Änderungen vom 20.10.2026 (Symbol allein auf der Zeile, Vorspann, Usage, `-o`/`-t`)

Hier wurde die .odt dem geänderten Assembler angepasst, nicht umgekehrt. Die Änderungen stehen alle in `ca82ass.c`. Geprüft mit einem aus dem Quelltext gebauten Linux-Binary: Alle 134 vorhandenen `.asm`-Dateien der Projektstruktur liefern mit der alten und der neuen Fassung dieselben `.obj`-, `.sym`-, `.log`-, `.dmp`- und `.mqo`-Dateien; bei den `.test`-Dateien ändert sich nur der Kopf (Nr. 2). Das Windows-Binary ist neu zu erzeugen (`make_c82ass.bat`).

Nr.	Stelle	Änderung in c82ass	Anmerkung
1	Zeile mit Symbol, aber ohne Operationscode (3.2, 3.3.7)	Pass 2 erhöhte den Befehlszähler auch für ein Symbol allein auf der Zeile, obwohl die Zeile kein Wort belegt. Alle folgenden Symbolwerte (`.sym`, `.log`) und damit Operanden, die auf sie verweisen, waren um 1 zu hoch; die Adressen im Listing stimmten. Jetzt erhält das Symbol die Adresse der nächsten Zeile, die Speicher belegt, und der Zähler bleibt unverändert. Vor `start` meldet der Assembler jetzt „Ein Label: … vor start ist nicht zulaessig.“ (vorher kam dort nur die irreführende Meldung, `start` fehle als erste Anweisung).	In den vorhandenen Quelltexten kommt das Muster nicht vor, deshalb ändert sich dort nichts. Tests: `Projekt_1/marke-allein` und `Projekt_1/marke-allein-vor-start` (Negativtest, endet mit Rückgabecode 16).
2	Kopf der `.test`-Datei (3.1)	Vorspannzeilen aus Kommentarzeilen des Quelltextes (Marken: siehe 3.1) ersetzen den bisher immer erzeugten Platzhalter-Kopf. Ohne Vorspannzeile entsteht weiterhin der Platzhalter. Mit `#STARTE` entfällt der Abschnitt `Tastenfolge:`.	`--pruefe-…`, `#INTERRUPT` und das Pragma `;#; LEITADRESSE` werden nicht übernommen. Bei 95 der 134 vorhandenen Quelltexte ändert sich dadurch der Kopf der `.test`-Datei, der übrige Teil der Datei nicht. Tests: `Projekt_1/test-vorspann`, `test-vorspann-standard`, `test-vorspann-starte`.
3	Aufruf: `-h`, `--help`, `--version` (2)	`-h` gibt die vollständige Usage aus (bisher nur `-p`, `-s` und `-WARN=SYM`), `--help` ist neu, `--version` gibt nur die Versionszeile aus und endet mit Rückgabecode 0.	`--version` wurde bisher wie eine unbekannte Option ignoriert (der Lauf verwendete die gespeicherten Parameter).
4	Schalter `-o` und `-t` (2)	Eigener Ablageort für `.obj` bzw. `.test` (Verzeichnis oder Dateiname, Regeln in Abschnitt 2).	Nicht in `projekdetails` gespeichert; `.log`, `.mqo`, `.sym` und `.dmp` bleiben im Projektverzeichnis.
5	`-WARN=SYS` (2)	Kein Code geändert: Der Assembler kennt nur `SYM`.	`SYS` stand fälschlich in der .odt und in der Ableitung; beide sind berichtigt.

6.4 Änderungen vom 21.10.2026 (Version 1.00-010: Warnungen, Schalter `-m`, `--xref`, Build)

Die Änderungen stehen in `ca82ass.c`, `ca82_standards.c`, `ca82_berichte.c`, `ca82_symbole.c`, `make_c82ass.sh` und `make_c82ass.bat`. Die .odt und das PDF sind dazu nicht nachgeführt. Geprüft mit `Regressionstest/regressionstest.sh`: alle 139 Programme liefern dieselben Ergebnisse wie vorher.

Nr.	Stelle	Änderung
1	Schalter `-m Makrodatei` (2)	Pfad der Makrodatei statt des festen Namens `c82ass.makros`. Mit `-m` wird nur dieser Pfad benutzt (keine Ersatzsuche in `Sourcecode/`); lässt sich die Datei nicht öffnen, endet der Lauf mit Rückgabecode 16 und einer Meldung mit Pfad und Ursache. `-m` ohne Wert ist ebenfalls ein Fehler (16). Ohne `-m` bleibt das Verhalten unverändert.
2	Compiler-Warnungen	Mit `-Wall -Wextra` meldet der Compiler keine Warnung mehr (vorher 29 × `-Waddress`, 11 × `-Wsign-compare`, ungenutzte Variablen und Parameter, wirkungslose Anweisung `literalCount;` in `pass1`). Die immer wahren Prüfungen auf `operation` und `operand` (Arrays, nie `NULL`) sind entfernt. Nicht aufgeräumt sind nur die `-Wshadow`-Hinweise (überdeckte Namen).
3	Zeichenketten ohne Endmarke	In `pass0` fehlte nach `strncpy` mit voller Puffergröße die abschließende 0 (`trimmedSp2`, `macroNameCleaned`); bei einem Feld von 256 oder mehr Zeichen wäre über das Ende gelesen worden. Zwei `sprintf` auf Puffer mit 1024 und 4096 Byte (Makroaufruf-Kommentar in `pass0`, Zeilenaufbau in `pass3`) sind zu `snprintf` mit Feldbegrenzung geworden. Der Puffer `maxParamStr` ist von 10 auf 16 Byte vergrößert.
3a	Symbol allein auf der Zeile (3.2)	`loeseMacroAuf` las bei leerem Operationscode ein Byte vor dem Puffer (`aktuellerOpCode[len - 1]` mit `len == 0`). Mit AddressSanitizer bei `marke_allein` und `marke_allein_vor_start` gefunden; geprüft mit AddressSanitizer/UBSan über alle 139 Programme ohne weiteren Fund.
4	`make_c82ass.sh`, `make_c82ass.bat`	Übersetzen mit `-O2 -Wall -Wextra`; `-fPIC` entfällt (zusammen mit `--static` überflüssig).
5	Regressionstest	Drei neue Negativtests für `-m` (`makro_ok`, `makro_fehlt`, `makro_ohne_wert`); Referenz für `Proj_FeldSortieren` aufgenommen.
6	Version	Die Versionsangabe steht jetzt an einer Stelle: `Sourcecode/ca82_version.h` (`ASM_VERSION`). Das Programm meldet „1.00-010“ (vorher „1.00“), in den geänderten Dateien steht sie in der Kopfzeile. Bei jeder Änderung werden Version und alle betroffenen Dateien nachgeführt. Die .odt trägt jetzt ebenfalls 1.00-010 und enthält die Änderungen als Abschnitt 6 „Änderungsverzeichnis“.
7	Schalter `--xref` (2)	Gibt nach fehlerfreiem Lauf eine Querverweisliste aus: je Symbol Wert, Definitionszeile (Name in Spalte 1) und alle Quelltextzeilen mit dem Namen als ganzem Wort (Kommentare ab `;` ausgenommen). Vom Assembler erzeugte Symbole fehlen; Vorkommen in Makroexpansionen und Makrodateien werden nicht erfasst. Ohne `--xref` ändert sich nichts. Tests: `xref_basis`, `xref_aus`.
8	Aufteilung von `ca82ass.c`	Erster Schritt: Usage, Versionszeile und die Berichte `-WARN=SYM` und `--xref` stehen jetzt in `ca82_berichte.c/.h`, der Typ `Symbol` in `ca82_symbole.h`, `gleichOhneGrossKlein` und `beginntOhneGrossKlein` in `ca82_standards.c`. `make_c82ass.sh` und `.bat` übersetzen `ca82_berichte.c` mit. `ca82ass.c` ist um rund 250 Zeilen kürzer; das Ergebnis des Assemblers ist unverändert (Regressionstest). Auch in der .odt (Abschnitt 6) nachgeführt.
9	Aufteilung von `ca82ass.c`, zweiter Schritt	Die Symbol-, Literal-, equ- und abs-Tabellen mit den Suchfunktionen (`isLiteralInTable`, `sucheInEquTabelle`, `sucheInAbsTabelle`, `isSymbol`, `getSymbolAddress` u. a.) und der Sortierung stehen in `ca82_symbole.c/.h`. Das Eintragen mit Fehlermeldung ins Listing (`addSymbol`, `addLiteral`) bleibt in `ca82ass.c`, weil es an den Listing-Arrays hängt. `ca82ass.c` hat jetzt rund 5.600 Zeilen (vorher 5.940). Ergebnis unverändert (Regressionstest, Sanitizer).

6.5 Änderungen vom 21.10.2026 (Pufferüberläufe, Fehlerorte für Pass 0 bis 2)

- Absturz (Rückgabecode 134) bei sehr langen Operationscodes und Labels behoben: alle `strcpy`/`strcat`/`sprintf` in `ca82ass.c` laufen über die begrenzten Helfer `STRKOPIE`/`STRANHANG` bzw. `snprintf` (`ca82_standards.h`). Zu lange Texte werden gekürzt statt überschrieben.
- Fehler aus Pass 0 bis 2 (doppeltes oder unzulässiges Label, Label vor `start` u. a.) erscheinen jetzt auch auf STDERR im Format `<datei>:<zeile>: Fehler: <text>`. Die Quellzeile wird über das Feld `srcLine` durch die Pässe getragen; bei Makroexpansionen gilt die Zeile des Makroaufrufs.
- Regressionstest: neue Negativtests `langes_label`, `lange_operation`, `doppeltes_label` (20 Negativtests insgesamt). Alle Programme und Negativtests laufen auch unter AddressSanitizer fehlerfrei.
- Das PDF und das Windows-Binary sind weiterhin nicht nachgeführt.

6.6 Änderungen vom 21.10.2026 (Version 1.00-011: keine `-Wshadow`-Hinweise mehr)

- Alle 49 Hinweise von `-Wshadow` sind beseitigt; der Quelltext übersetzt jetzt mit `-Wall -Wextra -Wshadow` ohne Meldung. `make_c82ass.sh` und `make_c82ass.bat` schalten `-Wshadow` ein.
- Umbenannt wurden nur Funktionsparameter und lokale Variablen, die globale Namen oder Variablen eines äußeren Blocks verdeckten (Endung `_p` für Parameter, `_in` für innere Variablen, z. B. `lineAsmAufgeloest_p`; in `ca82_symbole.c` `tabelle/anzahl` statt `symbolTable/symbolCount`; in den Meldungsmakros `em_len` statt `len`). Das Verhalten ändert sich nicht: Die Parameter verdeckten die globalen Namen schon vorher, die Funktionen benutzten bereits den Parameter.
- Prüfung: Regressionstest (139 Programme, 20 Negativtests) gleich, auch unter AddressSanitizer; alle String-Literale und Kommentare vor und nach der Umbenennung verglichen (unverändert).
- Hinweis für Änderungen: Die Meldungsmakros (`MELDE_FEHLER*`, `MELDE_FEHLER_P2`) greifen auf die **globalen** Zeilenfelder zu. Ein gleichnamiger Funktionsparameter würde sie überdecken; deshalb heißen solche Parameter jetzt eindeutig anders.
- Weiterhin offen: PDF und `.odt` (Stand 1.00-010) sowie das Windows-Binary sind nicht nachgeführt; die großen Funktionen (`loeseMacroAuf`, `pass2`, `pass3`) sind noch nicht zerlegt.

6.7 Änderungen vom 21.10.2026 (Version 1.00-012: Fehlerorte, Zeilengrenze, BOM)

Alle Änderungen stehen in `ca82ass.c`; Version und Kopfzeilen sind auf 1.00-012 gesetzt. Die .odt (Stand 1.00-009), das PDF und das Windows-Binary sind nicht nachgeführt. Geprüft mit `Regressionstest/regressionstest.sh`: 139 Programme unverändert, 26 Negativtests (6 neue), auch unter AddressSanitizer/UBSan.

Nr.	Stelle	Änderung
1	Absturz bei langen Quelltexten (3.2)	Ein Quelltext mit mehr als 8192 Zeilen (auch Kommentarzeilen) ließ den Assembler abstürzen (Rückgabecode 139). Jetzt gilt eine Grenze von 6144 Zeilen für Quelltext einschließlich Makroexpansion (die restlichen 2048 Einträge der festen Felder bleiben für Literalzeilen und Fehlermarken). Beim Überschreiten meldet der Assembler `<datei>:<zeile>: Fehler: Zu viele Zeilen: …` und endet mit Rückgabecode 16. Zusätzlich prüfen alle Schreibstellen der Zeilenfelder (Pass 1 bis 3, auch die Textzeichen von `dt`, `dto`, `dtm`) die Feldgrenze und melden „Das interne Zeilenfeld ist voll“ statt eines Überlaufs. Das größte vorhandene Programm hat 654 Zeilen.
2	UTF-8-Bytefolgemarke (BOM)	Eine BOM am Dateianfang (Windows-Editoren, z. B. Notepad) wird übersprungen. Vorher verband sie sich mit der ersten Zeile; die Meldung lautete irreführend „`start` fehlt als erste Anweisung“.
3	Makroaufruf fehlerhaft (Pass 0)	Fehler wie „erwartet 0 Parameter, aber 1 wurden übergeben!“ stehen jetzt zusätzlich mit Ort auf STDERR (`<datei>:<zeile>: Fehler: …`, Zeile des Makroaufrufs). Der bisherige Text „Warnung: Compilierung endete mit FEHLERn.“ bleibt.
4	Belegungsfehler (Pass 3)	„Adresse … doppelt belegt“ steht jetzt mit Ort auf STDERR (`<datei>:<zeile>: Fehler: …`); die Zeile ist die Quellzeile, die den Konflikt auslöst. Bei vom Assembler erzeugten Zeilen (Literale) steht nur `<datei>:`.
5	Grund bei ungültigem `dw`-Ausdruck	`dw 5/0` meldet „Operand '5/0' ist ungültig (Division durch 0)“, ein Überlauf bei `*` „(Ueberlauf bei der Multiplikation)“. Vorher fehlte der Grund.
6	Speicherfehler Makroaufruf	Beim Fehler „erwartet n Parameter“ waren die Felder `sp2` bis `sp4` der Fehlerzeile nicht belegt; die Weiterverarbeitung las über das Pufferende (mit AddressSanitizer beim neuen Test `makro_aufruf_parameter` gefunden). Die Meldungsmakros `MELDE_FEHLER` und `MELDE_WARNUNG` setzen jetzt alle Felder.
7	Schreibfehler	„Opertionscode“ in den Meldungen zu `start` ist „Operationscode“.
8	Regressionstest	Neue Negativtests: `makro_aufruf_parameter`, `belegung_ort`, `division_null`, `bom_ok`, `zu_viele_zeilen`, `zeilenfeld_voll` (26 insgesamt).

Hinweis für die .odt (Abschnitt 3.1, Fehlertabelle): Zeile „Makroaufruf fehlerhaft (Pass 0)“ und Zeile „Belegungsfehler in Pass 3“ nennen als STDERR-Ausgabe jetzt zusätzlich `<datei>:<zeile>: Fehler: …`. Abschnitt 3.2 („eine Zeile darf höchstens 255 Zeichen lang sein“) braucht den Satz zur Zeilengrenze (6144 Zeilen).

6.8 Änderungen vom 21.10.2026 (Version 1.00-013: Überlauf bei leerem `dt`, zwei Warnungen)

Alle Änderungen stehen in `ca82ass.c`; Version und Kopfzeilen sind auf 1.00-013 gesetzt. Der Regressionstest liefert unverändert dasselbe Ergebnis für alle 139 Programme; dazu 28 Negativtests (2 neue), auch unter AddressSanitizer/UBSan.

Nr.	Stelle	Änderung
1	Speicherfehler bei leerem `dt`, `dto`, `dtm`	Ein Textbefehl ohne Operand (`dt` allein auf der Zeile) las in `zaehleText` und `erzeugenText` ein Byte vor dem Operandenpuffer (Stack-Überlauf, mit AddressSanitizer gefunden). Die Abfrage auf das schließende `\` prüft jetzt zuerst, ob der Text leer ist. Dieselbe Absicherung gilt für das Entfernen der Endmarken bei Symbolnamen mit weniger als zwei Zeichen. Das Ergebnis für gültige Programme ist unverändert.
2	Warnung bei leerem Textbefehl	`dt`, `dto` oder `dtm` ohne Text erzeugt kein Wort. Das meldet der Assembler jetzt als `<datei>:<zeile>: Warnung: dt ohne Text: es wird kein Wort erzeugt.` auf STDERR. Rückgabecode bleibt 0, `.log` und `.dmp` ändern sich nicht.
3	Warnung bei leerem Programm	Eine leere oder nur aus Kommentaren bestehende `.asm` ergab bisher stillschweigend eine leere `.obj`. Jetzt erscheint auf STDERR `Warnung: Das Programm enthaelt keine Befehle und keine Daten; die Objektdatei ist leer.` Rückgabecode bleibt 0.
4	Regressionstest	Neue Negativtests `leeres_programm` und `dt_ohne_text` (28 insgesamt).
5	Zufallstest (Fuzzing)	Etwa 3700 zufällig veränderte Fassungen der vorhandenen Programme (Zeilen löschen, vervielfachen, kürzen, sehr lange Zeilen, Sonderzeichen, BOM, Zeilenende CR) liefen unter AddressSanitizer/UBSan. Nach der Korrektur 1 trat kein Absturz, keine Speicher- und keine UBSan-Meldung mehr auf; Rückgabecodes waren stets 0 oder 16.

Entscheidungen in dieser Version: Der Text „Warnung: Compilierung endete mit FEHLERn.“ bei Makrofehlern bleibt wie in der .odt. Die Pass-1-Fehler erscheinen weiter im Altformat „bei Operand: … in Zeile: …“ (steht in der .odt-Tabelle) zusätzlich mit `<datei>:<zeile>:`. `da a-b` wird weiter mit der Leitadresse verschoben; die Entscheidung dazu ist offen.

Hinweis für die .odt: Abschnitt 3.1 (Fehlertabelle) nennt für „Makroaufruf fehlerhaft (Pass 0)“ und „Belegungsfehler in Pass 3“ zusätzlich `<datei>:<zeile>: Fehler: …` auf STDERR. Abschnitt 3.2 braucht den Satz: „Der Quelltext darf einschließlich Makroexpansion höchstens 6144 Zeilen haben; Kommentarzeilen zählen mit. Bei Überschreitung meldet der Assembler `<datei>:<zeile>: Fehler: Zu viele Zeilen …` und endet mit Rückgabecode 16.“ Außerdem gehören die Warnungen aus Nr. 2 und 3 in die Meldungstabelle.

6.9 Änderungen vom 21.10.2026 (Version 1.00-014: `pass3` zerlegt)

Reine Umstrukturierung in `ca82ass.c`, das Verhalten ist unverändert.

Nr.	Stelle	Änderung
1	`pass3`	Die Funktion hatte rund 490 Zeilen und steht jetzt bei 168. Jede Anweisung hat eine eigene Funktion: `pass3Start`, `pass3Equ`, `pass3Spa`, `pass3Abs`, `pass3Dw`, `pass3Da`, `pass3Db`, `pass3Dbn`, `pass3Text` (für `dt`, `dto`, `dtm`) und `pass3Befehl` (normale Maschinenbefehle). Die Schleife in `pass3` wählt nur noch die Funktion aus.
2	Gemeinsamer Zustand	Befehlszähler, relative Adresse, Dateien und Spaltenformat stehen in der Struktur `Pass3Zustand` statt in lokalen Variablen von `pass3`.
3	Verschachtelte Funktionen	`istInDreua` und `getTrimmed` waren in `pass3` verschachtelt (GCC-Erweiterung) und sind jetzt normale `static`-Funktionen; das erleichtert die Übersetzung mit anderen Compilern. `getTrimmed` kürzt zu lange Texte, statt den Puffer zu überschreiten.
4	Prüfung	Regressionstest unverändert: 139 Programme, 28 Negativtests, auch unter AddressSanitizer/UBSan. Zusätzlich Vergleich der alten und der neuen Fassung auf 700 zufällig veränderten Quelltexten: Rückgabecode, STDOUT, STDERR sowie `.obj`, `.mqo`, `.sym`, `.dmp`, `.test` und `.log` waren in allen Fällen identisch.

Hinweis: `loeseMacroAuf` und `pass2` wurden in 1.00-015 zerlegt (6.10).

6.10 Änderungen vom 21.10.2026 (Version 1.00-015: `pass2` und `loeseMacroAuf` zerlegt, Speicherfehler behoben)

Nr.	Stelle	Änderung
1	`pass2`	Von rund 600 auf etwa 70 Zeilen. Jede Anweisung hat eine eigene Funktion: `pass2Start`, `pass2Equ`, `pass2Spa`, `pass2Abs`, `pass2Db` (für `db`, `dbn`), `pass2Text` (für `dt`, `dto`, `dtm`), `pass2Dz`, `pass2DwDa` (für `dw`, `da`), `pass2Ende`, dazu `pass2Befehl`, `pass2PruefeUeberlauf` und die Ausgabe von Symbol- und Adresstabelle. Der Zustand steht in der Struktur `Pass2Zustand`. Die Anweisungen werden wie bisher nacheinander und unabhängig voneinander geprüft. Die verschachtelten Funktionen (`copyAndIncrementLineElements2`, `replaceOrigOperation`, `resolveSymbols`) sind jetzt normale `static`-Funktionen. Tote Zweige (`if (1)`, nie erreichte `else`-Teile) sind entfernt.
2	`loeseMacroAuf`	Von rund 600 auf etwa 90 Zeilen. Prüfung der Makrodefinition (`pruefeMakroDefinition`, `pruefeMakroZeile`, `pruefePrZeile`), Übernahme der Zeilen, Ersetzen der Parameter, Auswertung von `$IF`/`$ELIF`/`$ENDIF` (`makroBedingungenAuswerten`), `!$wdhl` und Label des Aufrufs haben je eine Funktion. Die Fehler eines Aufrufs stehen in der Struktur `MakroFehler`.
3	Fehler: Heap-Überlauf bei `$IF`-Fehlern	Meldete `macroIfAufloesen` einen Fehler (z. B. `$IF` geschachtelt oder falsche Parameterzahl), zählte die Schleife in `loeseMacroAuf` das Zeilenende mit und las hinter das Zeilenfeld (mit AddressSanitizer gefunden). Die Schleife läuft jetzt nur über die vorhandenen Zeilen. An der Fehlerstelle bleibt wie bisher eine Leerzeile. Gültige Programme sind nicht betroffen.
4	Fehler: Stack-Überlauf in `$IF`	In `macroIfAufloesen` waren die Felder für `|`/`&` nur 2 Byte groß; ein längeres Wort an dieser Stelle überschrieb den Stack. Alle Felder haben jetzt 128 Byte und werden mit Feldbreite gelesen. Ein zu langer Operator ergibt die Meldung „Der 4. Parameter muss | oder & sein.“
5	Fehler: Speicherlecks	`loeseMacroAuf` gab das Ergebnisfeld bei einem Namen, der kein Makro ist, und das Zwischenfeld bei jedem Makroaufruf nicht frei. Beides wird freigegeben; unter AddressSanitizer mit Leak-Prüfung ist der Regressionstest jetzt frei von Meldungen.
6	Fehler: Makrofehler gingen bei genau 64 Fehlern verloren	Bei der Obergrenze von 64 Fehlern wurde keiner davon an die Ausgabe angehängt, der Lauf endete aber mit Fehler. Jetzt werden alle 64 ausgegeben.
7	Kleinere Absicherungen	`pass2`: Zeilenpuffer ist initialisiert (war nicht initialisiert); fehlgeschlagenes Öffnen von `.log` oder `.sym` wird gemeldet (früher Absturz). `loeseMacroAuf`: Makroname ohne Zeichen und zu lange Namen in Ausdrücken der späten Symbole führen nicht mehr zu Zugriffen außerhalb des Puffers; die erzeugte `equ`-Zeile eines Makrolabels hat eine definierte Quellzeile (0).
8	Regressionstest	Drei neue Negativtests: `makro_if_geschachtelt`, `makro_if_parameterzahl`, `makro_if_operator_lang` (31 insgesamt).
9	Prüfung	139 Programme und 31 Negativtests unverändert bzw. grün, auch unter AddressSanitizer/UBSan mit Leak-Prüfung. Vergleich alte gegen neue Fassung auf 1500 zufällig veränderten Quelltexten: Rückgabecode, STDOUT, STDERR und alle sechs Ausgabedateien identisch, außer in den 11 Fällen, in denen die alte Fassung die Speicherfehler aus Nr. 3 und 4 hatte. Zufallstest unter AddressSanitizer: 1900 Läufe ohne Meldung.

Offen: `pass1`, `pass0` und `main` haben noch jeweils rund 200 Zeilen; der Quelltext liegt weiter in einer Datei `ca82ass.c`.

6.11 Änderungen vom 21.10.2026 (Version 1.00-016: `da` mit nicht definiertem Symbol)

Nr.	Stelle	Änderung
1	Fehler: `da` mit unbekanntem Symbol	`x: da nirgends` (oder `da nirgends+spaeter`) wurde ohne jede Meldung mit dem Wert 0 übersetzt, der Lauf endete mit Rückgabecode 0 und erzeugte eine `.obj`. Ursache: Die späte Auflösung der Symbole (Pass 2) erkannte das fehlende Symbol, setzte aber das Fehlerkennzeichen nicht und trug die Meldung an einer falschen Stelle ein. Jetzt meldet der Assembler `<datei>:<zeile>: Fehler: Symbol 'nirgends' ist nicht definiert.` bei der Zeile, in der das Symbol (hier `x`) definiert wird, schreibt die Fehlermarke ins Listing, erzeugt keine `.obj` und endet mit Rückgabecode 16. Bei `dw` kam die Meldung schon vorher aus Pass 3.
2	Symboltabelle	Neues Feld `aufgeloestIndex` (Zeile der Definition im aufgelösten Listing). Es wird nur für diese Meldung gebraucht.
3	Regressionstest	Neuer Negativtest `da_symbol_fehlt` (32 insgesamt). Die 139 Programme sind unverändert. Vergleich alte gegen neue Fassung auf 500 zufällig veränderten Quelltexten: Unterschiede nur bei der neuen Meldung (3 Fälle) und bei den Speicherfehlern der alten Fassung (2 Fälle). Zufallstest unter AddressSanitizer: 400 weitere Läufe ohne Meldung.

Damit ist Punkt 1 aus „Was man noch verbessern könnte“ erledigt: Der Wert -1 für ein nicht gefundenes Symbol kam in der Praxis nie vor, der eigentliche Fehler war das fehlende Fehlerkennzeichen.

6.12 Änderungen vom 21.10.2026 (Version 1.00-017: Fehlerort bei Fehlern in der Makrodatei, Prüfskript, CI)

Nr.	Stelle	Änderung
1	Fehler in `c82ass.makros`	Ein Fehler in der Definition eines Makros der Makrodatei (z. B. ein falsches `$IF`) wurde bisher nur mit dem Ort des Makroaufrufs im Quelltext gemeldet; wo der Fehler in der Makrodatei steht, ließ sich nicht erkennen. Jetzt hängt der Assembler an jede solche Meldung die Herkunft an, z. B. `t.asm:3: Fehler: BAD $IF benötigt 3, 7, 11 oder 15 Parameter ... [Makro BAD, definiert in c82ass.makros:7]`. Die Zeilennummer ist die der `MACRO`-Zeile des Makros. Die Angabe steht auch in der Fehlerzeile des Listings. Makros aus dem Quelltext sind nicht betroffen (dort ist der Ort ohnehin der Quelltext). Die Struktur `macroDefinitionen` hat dafür die Felder `defDatei` und `defZeile`.
2	Regressionstest	Neuer Negativtest `makrodatei_fehlerort` (33 insgesamt). Dazu kann ein Negativtest jetzt eine eigene Makrodatei `<name>.makros` mitbringen, die zusammen mit der `.asm` in das Arbeitsverzeichnis kopiert und mit `args -m w/negativ/<name>.makros` verwendet wird.
3	Prüfskript	Neu: `Regressionstest/pruefe_alles.sh`. Es übersetzt mit `-Wall -Wextra -Wshadow -Werror` (jede Warnung ist ein Fehler), führt den Regressionstest aus und wiederholt ihn mit einem Binary unter AddressSanitizer und UBSan mit Leak-Prüfung. Rückgabecode 0 = alles in Ordnung, sonst die Nummer des fehlgeschlagenen Schritts. Mit `--ohne-asan` entfällt der dritte Schritt.
4	CI	Neu: `.github/workflows/assembler.yml` führt `pruefe_alles.sh` bei jedem Push und Pull Request aus, soweit sie `Assembler/` betreffen.
5	Prüfung	139 Programme und 33 Negativtests grün, auch unter AddressSanitizer/UBSan mit Leak-Prüfung; Übersetzung ohne Warnung (auch mit `-Werror`).

Offen: `pass0`, `pass1` und `main` haben noch jeweils rund 200 Zeilen; der Quelltext liegt weiter in einer Datei `ca82ass.c` (das Makro-Modul `ca82_makros.c` ist der nächste Schritt). Entscheidung zu `da a-b` mit der Leitadresse und das neue Windows-Binary stehen noch aus; das PDF ist nicht nachgeführt.

6.13 Änderungen vom 21.10.2026 (Version 1.00-018: Makro-Modul herausgelöst)

Nr.	Stelle	Änderung
1	Aufteilung von `ca82ass.c`	Der Makro-Präprozessor steht jetzt in `ca82_makros.c`/`.h` (Einlesen der Makrodatei, Quelltextmakros, `loeseMacroAuf`, `macroSprung1Aufloesen`, `macroIfAufloesen`, `printAllMacros`). Gemeinsame Typen (`lineElements`, `macroDefinitionen`, `MacroLabel`), Konstanten und die Meldemakros `MELDE_FEHLER`/`MELDE_WARNUNG` stehen in `ca82_typen.h`. `ca82ass.c` schrumpft von 5.822 auf rund 4.270 Zeilen.
2	Pfad der Makrodatei	Der Pfad (Option `-m`) gehört dem Makromodul; `ca82ass.c` setzt ihn über `makroDateiSetzen()` und liest ihn über `makroDateiName()`.
3	Build	`make_c82ass.sh`, `make_c82ass.bat`, `Regressionstest/pruefe_alles.sh` und die Sanitizer-Zeile in `Regressionstest/LIESMICH.md` übersetzen `ca82_makros.c` mit.
4	Verhalten	Unverändert: Es ist ein reines Umordnen des Quelltexts. 139 Programme und 33 Negativtests liefern byte-gleiche Ergebnisse; Übersetzung ohne Warnung (`-Werror`), auch unter AddressSanitizer/UBSan.

Offen: `pass0`, `pass1` und `main` (je rund 200 Zeilen) sind noch zu zerlegen; Neues Windows-Binary und Nachführung des PDF stehen aus (`da a-b`: entschieden, siehe 6.14).

6.14 Änderungen vom 21.10.2026 (Version 1.00-019: `da` verlangt eine relative Adresse)

Nr.	Stelle	Änderung
1	`da` und `@`-Literale (3.3.4)	Entscheidung: Bei `da` muss immer eine relative Adresse entstehen. Konstanten und konstante Ausdrücke sowie paarweise Differenzen relativer Adressen (`a-b`) sind unzulässig; es muss mindestens ein relativer Adressanteil übrig bleiben, z. B. `a-b+c`. Geprüft wird in Pass 2 (`pass2DwDa`, Hilfsfunktion `adressausdruckGrad`) am Quelltext des Operanden, also auch bei Vorwärtsreferenzen. Fehler erscheinen mit Datei und Zeile auf STDERR und im Listing; Rückgabecode 16.
2	Relative Adresse in `*` und `/`	`da a*2` und `da a/2` sind unzulässig (eine Adresse ist nicht multiplizierbar); das entspricht der Regel bei `dw`.
3	Sternadresse	`da *` bleibt unverändert zulässig.
4	Literale	Ein fehlerhaftes `@`-Literal (z. B. `@5`) wird an der Stelle gemeldet, an der der Assembler die Literal-`da`-Zeile am Programmende erzeugt, nicht an der Zeile mit dem Literal.
5	Regressionstest	Neue Negativtests `da_relativ_fehler` und `da_relativ_ok` (35 insgesamt). Das Beispiel `Claude_bsp_reassemblierung/asm_0001_bsp_5_16` verwendet `da d2100` mit `d2100: abs 02100` (eine Konstante) und wird jetzt abgewiesen; seine Referenz wurde bewusst auf das neue Verhalten (Rückgabecode 16, nur .log/.sym/.rc) gesetzt. Das Programm ist anzupassen.
6	Prüfung	139 Programme und 35 Negativtests grün; Übersetzung ohne Warnung (`-Werror`), auch unter AddressSanitizer/UBSan.

6.15 Änderungen vom 21.10.2026 (Version 1.00-020: Quelltext in Module zerlegt, `pass0`, `pass1` und `main` zerlegt)

Nr.	Stelle	Änderung
1	Aufteilung von `ca82ass.c`	Die Datei `ca82ass.c` (4332 Zeilen) entfällt. Ihr Inhalt steht in zwölf Modulen: `ca82_main.c` (Optionen, Dateinamen, Ablauf der Pässe), `ca82_quelle.c` (Einlesen, Pragma, Wiederholungsadressen), `ca82_listing.c` (Listing, `.dmp`), `ca82_pass0.c`, `ca82_pass1.c`, `ca82_pass2.c`, `ca82_pass3.c`, `ca82_ausdruck.c` (Werte- und Adressausdrücke), `ca82_codierung.c` (Operationscodes, Text, `dz`, Umschlüsselung), `ca82_testdatei.c` (`.test`), `ca82_hilfen.c` (Zeilenfelder, Meldungen, Textwerkzeuge) und `ca82_zustand.c` (globale Zustandsvariablen). Die gemeinsamen Deklarationen stehen in `ca82_intern.h`.
2	Globale Variablen	Die Dateinamen für `.log`, `.obj`, `.mqo`, `.sym`, `.test` und `.dmp` sowie Projektverzeichnis und Quellcodedatei sind jetzt `static` in `ca82_main.c`; die übrigen Module bekommen sie als Parameter. Die ungenutzte Variable `oktalzahl` entfällt.
3	`pass0`	Zerlegt in `pass0PruefeMakrodefinitionen`, `pass0PruefeMakroEnden`, `pass0EinDurchlauf`, `pass0ZeileAufloesen` und `pass0EnthaeltMakroaufruf`; die Schleife mit dem Merker `wdhlg` ist eine einfache Schleife.
4	`pass1`	Zerlegt in `pass1PruefeLiteralOperand`, `pass1LiteralErsetzen`, `pass1Equ`, `pass1Abs`, `pass1Sonstige`, `pass1SchreibeLog` und `pass1HaengeLiteraleAn`.
5	`main`	Zerlegt in `bereinigeParameter`, `pruefeOptionen`, `bildeDateinamen`, `entferneAlteErgebnisse`, `assembliere` und `meldeErfolg`; die dreifach wiederholte Freigabe der Makros steht in `gibMakrosFrei`. Eine Quellcodedatei ohne Punkt im Namen führt nicht mehr zu einem ungültigen Zeigerabzug.
6	Ausgabedateien	Kann eine Ausgabedatei (`.log`, `.sym`, `.obj`, `.mqo`, `.dmp`) nicht geschrieben werden, meldet der Assembler jetzt `datei: Fehler: Datei kann nicht geschrieben werden: <Ursache>` und bricht mit Rückgabecode 16 ab. Früher wurde ein NULL-Zeiger weiterbenutzt.
7	Build	`make_c82ass.sh`, `make_c82ass.bat`, `Regressionstest/pruefe_alles.sh` und die Sanitizer-Zeile in `Regressionstest/LIESMICH.md` übersetzen die neuen Module.
8	Verhalten	Sonst unverändert. 139 Programme und 35 Negativtests liefern byte-gleiche Ergebnisse; Übersetzung ohne Warnung (`-Werror`), auch unter AddressSanitizer/UBSan.

Offen: Neues Windows-Binary und Nachführung des PDF stehen aus. Die Dokumentation 6.1 bis 6.14 nennt `ca82ass.c`; das ist der Stand der jeweiligen Version.

6.16 Änderungen vom 21.10.2026 (Version 1.00-021: zentrale Fehlerausgabe mit Obergrenze)

Nr.	Stelle	Änderung
1	Neues Modul `ca82_fehler.c`	Alle Fehlermeldungen mit Ort (`datei:zeile: Fehler: text`) auf STDERR laufen durch `fehlerMitOrt()`. Das betrifft die Fehlermarken aus Pass 0 bis 2, Fehler aus Pass 3 (nicht definierte Symbole, Adresse doppelt belegt) und „Zu viele Zeilen“.
2	Obergrenze	Es werden höchstens 20 Fehlermeldungen ausgegeben. Weitere werden gezählt; am Programmende (auch bei Abbruch mitten in einem Pass) steht eine Zeile `datei: 25 Fehler insgesamt, 5 weitere Fehlermeldungen unterdrückt (Obergrenze 20)`. Bei bis zu 20 Fehlern bleibt die Ausgabe unverändert. Der Rückgabecode ist weiterhin 16.
3	Listing	Unverändert: Das Listing (STDOUT, `.log`, `.dmp`) enthält weiterhin alle Fehlermarken.
4	Regressionstest	Neuer Negativtest `viele_fehler` (36 insgesamt).
5	Prüfung	139 Programme und 36 Negativtests grün; Übersetzung ohne Warnung (`-Werror`), auch unter AddressSanitizer/UBSan.

Nicht umgesetzt: eine vollständig zentrale Fehlerliste, die Abbrüche mitten im Pass ersetzt. Dafür müssten die Pässe ohne `exit()` weiterlaufen können; das hängt an der Umstellung auf eine gemeinsame Programmstruktur statt globaler Felder.

6.17 Änderungen vom 21.10.2026 (Version 1.00-022: `projekdetails` abgesichert, Referenz bereinigt, Makefile)

Nr.	Stelle	Änderung
1	`projekdetails` (`ca82_standards.c`)	**Absturz behoben:** Konnte die Temp-Datei `temp_projekdetails` nicht angelegt werden (Arbeitsverzeichnis nicht beschreibbar, Datei gesperrt, Verzeichnis gleichen Namens), endete der Assembler mit Speicherzugriffsfehler (Rückgabecode 139). Jetzt gibt es einmal die Warnung „Die Datei 'projekdetails' kann nicht aktualisiert werden …“, der Lauf geht normal weiter (`-p`/`-s` stehen ohnehin in der Kommandozeile).
2	`projekdetails`	Das Schreiben der Datei steht jetzt an einer Stelle (`projektdetailsZeileSetzen`) statt dreimal. Die alte Datei bleibt bei jedem Fehler (auch Schreibfehler, Platte voll) unverändert; es bleiben keine offenen Dateien zurück. Nicht initialisierte Zeile bei leerer `projekdetails` und das Lesen hinter das Zeilenende bei einer Zeile ohne Wert behoben.
3	Referenz	Für `Claude_bsp_reassemblierung/asm_0001_bsp_5_16` lagen noch `.obj`, `.mqo`, `.dmp` und `.test` aus der Zeit vor 1.00-019 (`da` verlangt eine relative Adresse). Das Programm endet seitdem mit Rückgabecode 16 und schreibt diese Dateien nicht mehr; die veralteten Referenzdateien wurden gelöscht. Vorher meldete der Regressionstest hier eine Abweichung.
4	Prüfskript	Neue Schlüsselwörter `vorher-verz` und `vorher-datei` für Negativtests (Vorbedingungen im Arbeitsverzeichnis). Neuer Negativtest `projekdetails_nicht_schreibbar` (37 insgesamt).
5	Build	Neues `Makefile` (`make`, `make test`, `make pruefen`, `make clean`) mit einer Quelldatei-Liste.
6	Prüfung	139 Programme und 37 Negativtests grün; Übersetzung ohne Warnung (`-Werror`), auch unter AddressSanitizer/UBSan.

Weiterhin offen: Windows-Binary, Nachführen von PDF und `.odt` (liegt bei Promat), Fehlerliste ohne `exit()`-Abbrüche (gemeinsame Programmstruktur), Warnungen für Sprünge auf Datenwörter und nicht erreichbaren Code.

6.18 Änderungen vom 21.10.2026 (Version 1.00-023: Rekursionsfehler ohne Abbruch)

Nr.	Stelle	Änderung
1	Makroauflösung (`ca82_makros.c`)	Ein rekursiver Makroaufruf beendete bisher den ganzen Lauf mit `exit()`, ohne Listing und ohne die übrigen Fehler. Jetzt endet nur die Übernahme dieses Makros; Pass 0 läuft weiter, das Listing enthält alle Fehlermarken, und die weiteren Fehler des Quelltexts werden mit Datei und Zeile gemeldet. Rückgabecode weiterhin 16.
2	Regressionstest	Neuer Negativtest `makro_rekursion` (38 insgesamt).
3	Bestandsaufnahme der Abbrüche	In Pass 1 bis 3 gibt es kein `exit()` mehr. Übrig sind nur Abbrüche, nach denen ein Weiterarbeiten nichts bringt: Quell-/Makro-/Ausgabedatei nicht zu öffnen, Speichermangel, Zeilenfeld voll bzw. „Zu viele Zeilen“, Makrodefinitionsfehler (alle werden vorher gezählt und gemeldet) und das Ende von Pass 0 nach vollständiger Auflösung (das Listing wird dort schon ausgegeben). Die Pässe sind stufenweise: Fehler aus Pass 1 verhindern Pass 2, weil Pass 2 gültige Pass-1-Ergebnisse braucht. Eine gemeinsame Programmstruktur statt globaler Felder ist dafür nicht mehr nötig.
4	Prüfung	139 Programme und 38 Negativtests grün; `-Werror`, AddressSanitizer/UBSan sauber.

6.19 Änderungen vom 21.10.2026 (Version 1.00-024: Warnungen für Sprungziele und nicht erreichbaren Code)

Nr.	Stelle	Änderung
1	`-WARN=SPR` (neu, `ca82_fluss.c`)	Meldet Sprungbefehle, deren Ziel ein Datenwort (`dw` ungleich 0, `da`) oder ein unbelegtes Wort ist. Ausnahmen und Einzelheiten stehen in Abschnitt 2.
2	`-WARN=TOT` (neu)	Meldet Befehle hinter einem unbedingten Sprung, auf die nichts verweist (toter Code). Ausnahmen: Sprungtabellen und zur Laufzeit überschriebene Sprungbefehle.
3	`-WARN=ALLE` (neu)	Schaltet `SYM`, `SPR` und `TOT` gemeinsam ein. Ein unbekannter Wert nennt jetzt alle erlaubten Werte (`SYM, SPR, TOT, ALLE`).
4	Ablauf	Pass 3 vermerkt je Trommeladresse, ob dort ein Befehl, ein Datenwort, ein Platzhalter oder ein Ausgabebefehl steht, und bei Sprüngen das Ziel. Die Auswertung läuft nach fehlerfreiem Pass 3 und nur mit der Option. Ohne die Optionen ist die Ausgabe byte-gleich zu 1.00-023 (alle 139 Programme gegen die Referenz).
5	Erfahrung an den 139 Programmen	Mit `-WARN=SPR,TOT` melden 7 Programme etwas. Die meisten Treffer stammen aus den Buchbeispielen zum Lochbanddruck (`bsp_5_15_…`, `bsp_5_17_oktaldruck`), deren Druckbefehle über eine Wertetabelle ausgewählt werden; dort ist der Hinweis zu erwarten. Echte Funde sind u. a. `Projekt_1/bsp_5.15` (Befehl `ANWAHL LSL2` hinter `000 eingabe`) und `Proj_FeldSortieren/code_wdhl_bef` (`WDHL …` hinter `000 loop1`). Beim ersten Entwurf meldete `SPR` auch `db`-Zellen und `dz`-Ziele sowie `TOT` Sprungtabellen; diese Fälle sind ausgenommen, weil sie auf der C8205 übliche Technik sind.
6	Regressionstest	Acht neue Negativtests (46 insgesamt): `warn_spr_datenwort`, `warn_spr_ok`, `warn_spr_leer`, `warn_tot`, `warn_tot_ok`, `warn_alle`, `warn_unbekannt`, `warn_ohne_option`.
7	Build	`ca82_fluss.c` steht in `Makefile`, `make_c82ass.sh`, `make_c82ass.bat` und `Regressionstest/pruefe_alles.sh`.
8	Prüfung	139 Programme und 46 Negativtests grün; `-Werror`, AddressSanitizer/UBSan sauber, auch mit `-WARN=ALLE --xref` über alle 139 Programme.

6.20 Änderungen vom 21.10.2026 (Version 1.00-025: Berichte im `.log`, Rekursionsfehler mit Ort, `make fuzz`)

Nr.	Stelle	Änderung
1	Berichte im `.log` (`ca82_main.c`, `ca82_standards.c`, `ca82_hilfen.c`, `ca82_berichte.c`, `ca82_fluss.c`)	Die Berichte von `-WARN=SYM`, `-WARN=SPR`, `-WARN=TOT` und `--xref` stehen jetzt zusätzlich am Ende der `.log`-Datei, neben der Symboltabelle, ohne Farbcodes. Dafür gibt es `berichtPrintf` (STDOUT und `.log`); `printWarnung` und `meldeWarnungOrt` schreiben ebenfalls ins `.log`, solange es für die Berichte geöffnet ist. Ohne diese Optionen ist die `.log` byte-gleich zu 1.00-024 (alle 139 Programme gegen die Referenz).
2	Rekursionsfehler (`ca82_makros.c`)	Die Meldung „Rekursiver Macro-Aufruf von: X“ nennt jetzt das Makro und den Ort: bei Makros der Makrodatei Datei und Zeile der `MACRO`-Anweisung, bei Makros im Quelltext die Zeile des Selbstaufrufs, z. B. `Rekursiver Macro-Aufruf von: REK [Makro REK ruft sich selbst auf, im Quelltext Zeile 3]`.
3	`make fuzz`	Baut ein Binary mit AddressSanitizer/UBSan und startet `Regressionstest/fuzz.py` (`FUZZ_N` Läufe, `FUZZ_SEED` Startwert). `fuzz.py` endet mit Rückgabecode 1, wenn etwas gefunden wurde. Neu: `.github/workflows/assembler-fuzz.yml` lässt den Zufallstest nachts laufen und sichert Funde als Artefakt.
4	Regressionstest	Neue Schlüsselwörter `log-da` und `log-weg` prüfen den Inhalt der `.log`. Zwei neue Negativtests `warn_log` und `warn_log_aus` (48 insgesamt); `makro_rekursion` prüft den Ort.
5	Offen	Die Opcode-Tabelle und `--xref` für Makroexpansionen wurden in 1.00-026 entschieden (6.21).

6.21 Änderungen vom 21.10.2026 (Version 1.00-026: Opcode-Tabelle entfernt, `--xref` mit Makroexpansionen)

Nr.	Stelle	Änderung
1	Opcode-Tabelle (entfällt)	`Sourcecode/c8205_op_codes.c` ist gelöscht, ebenso die Suche in der Tabelle in `getOperationCode` (`ca82_codierung.c`) und die Abhängigkeit im `Makefile`. Die Mnemonics `add`, `sub`, `ld`, `jmp`, `stopp` und `nop` (Platzhalterwerte, die nicht zum Befehlssatz passten) sind damit ungültige Operationscodes. Gültig bleiben nur dreistellige Oktalcodes. Keines der 139 Programme benutzt sie (Referenz unverändert). Abschnitt 3 der Doku entsprechend geändert.
2	`--xref` und Makros (`ca82_berichte.c`, `ca82_pass0.c`)	Entscheidung: Die expandierten Makrozeilen stehen im Listing, deshalb zählen sie mit. Pass 0 meldet bei `--xref` die Operanden jeder Makroexpansion; die Querverweisliste führt ein dort verwendetes Symbol in der Zeile des Makroaufrufs auf (nur die Operanden, nicht Marken und Kommentare; je Zeile einmal, Zeilennummern jetzt sortiert). Beispiel: Ruft ein Makro `000 ziel` auf und wird in den Zeilen 5 und 6 aufgerufen, steht `ziel` mit `3 5 6` (3 = Makrodefinition im Quelltext). Ohne `--xref` ändert sich nichts.
3	Regressionstest	Neue Negativtests `xref_makro` und `mnemonic_entfallen` (50 insgesamt).

6.22 Änderungen vom 21.10.2026 (Version 1.00-027: Schalter `--ABSPR`)

Nr.	Stelle	Änderung
1	Neuer Schalter `--ABSPR` (`ca82_main.c`, `ca82_codierung.c`, `ca82_pass3.c`, `ca82_berichte.c`)	Schreibt statt `.obj` die binär codierte Lochstreifendatei `.kep` für das KEP (Abschnitt 3.1a). Die Rückumwandlung mit `c8205_lochstreifen_umwandlung.py --IN-TXT` liefert die erwartete Zeichenfolge (geprüft mit `Claude/betrag.asm`). Ohne den Schalter ist die Ausgabe unverändert.
2a	Warnung Bahn 0000 bis 0037	Bei `--ABSPR` Warnung, wenn ein Wort auf eine Zelle von 0002 bis 0037 (das KEP selbst) geladen wird. Die Zellen 0000 und 0001 sind erlaubt (ab 1.00-029; zuvor warnte jeder Leitbefehl auf 0000 bis 0037).
2	Pragma `LEITADRESSE`	Bei `--ABSPR` gilt das Pragma zuerst; ohne verbleibendes K1=01 sowie bei K1=02..09 (nicht umgesetzt) erscheint je eine Warnung.
3	Offen	Der Regressionstest meldet für `Projekt_2/pj2-bis_minimalprogramm.asm` eine Abweichung (`.obj`, `.mqo`, `.dmp`); sie besteht auch mit dem unveränderten Quelltext der Version 1.00-026 (die Referenz stimmt dort nicht mit dem Quelltext überein). Nicht geändert. Für `--ABSPR` gab es zu diesem Zeitpunkt noch keinen Regressionstest (inzwischen vorhanden, siehe 6.25).

6.23 Änderungen vom 22.10.2026 (Version 1.00-028: Warnung bei `--ABSPR` mit `LEITADRESSE 00` und Startadresse)

Nr.	Stelle	Änderung
1	Warnung `start` bei `--ABSPR`	Neue Warnung bei `--ABSPR` mit Pragma `LEITADRESSE 00` und `start <adresse>` ungleich 0: Die Startadresse wird nicht auf die Operanden addiert. Regel: relativ geschriebener Quelltext mit `start <adresse>` ohne Pragma; absolut geschriebener Quelltext mit Pragma `LEITADRESSE 00` und `start` ohne Operand. (Quelle: Änderungsverzeichnis der .odt)

6.24 Änderungen vom 22.10.2026 (Version 1.00-029: Warnung zur KEP-Bahn eingeschränkt)

Nr.	Stelle	Änderung
1	Warnung Bahn bei `--ABSPR`	Die Warnung zur KEP-Bahn kommt nur noch, wenn ein Wort auf eine Zelle von 0002 bis 0037 (das KEP selbst) geladen wird. Die Sprungzellen 0000 und 0001 sind erlaubt und warnen nicht (Festlegung des Projektinhabers); vorher warnte jeder Leitbefehl auf 0000 bis 0037. (Quelle: Änderungsverzeichnis der .odt; siehe auch 6.22, Nr. 2a)

6.25 Änderungen vom 22.10.2026 (Version 1.00-030: Regressionstest für `--ABSPR`)

Der Eintrag „Version 1.00-030 (22.10.2026)“ im Änderungsverzeichnis der .odt hat noch keinen Text. Die folgenden Angaben stammen aus `Regressionstest/LIESMICH.md` und nicht aus der .odt.

Nr.	Stelle	Änderung
1	Regressionstest	Jedes Programm wird ein zweites Mal mit `--ABSPR` assembliert; verglichen werden der Lochstreifen `.kep` und der Rückgabecode (`.rck`) mit `referenz/`.
2	Negativtests	Vier neue Negativtests: `abspr_ok`, `abspr_bahn_warnung`, `abspr_start_warnung`, `abspr_ohne_schalter` (insgesamt 54). `negativ/version.erwartet` erwartet 1.00-030.
3	Referenz	Die Referenz von `namen_raten` und `pj2-bis_minimalprogramm` (`.obj`, `.dmp`) ist nach dem Pragma `LEITADRESSE 00` neu geschrieben. Damit entfällt die in 6.22, Nr. 3 genannte Abweichung.